

Трансформеры (I): GPT-n, BERT, BART, T5...

Recap

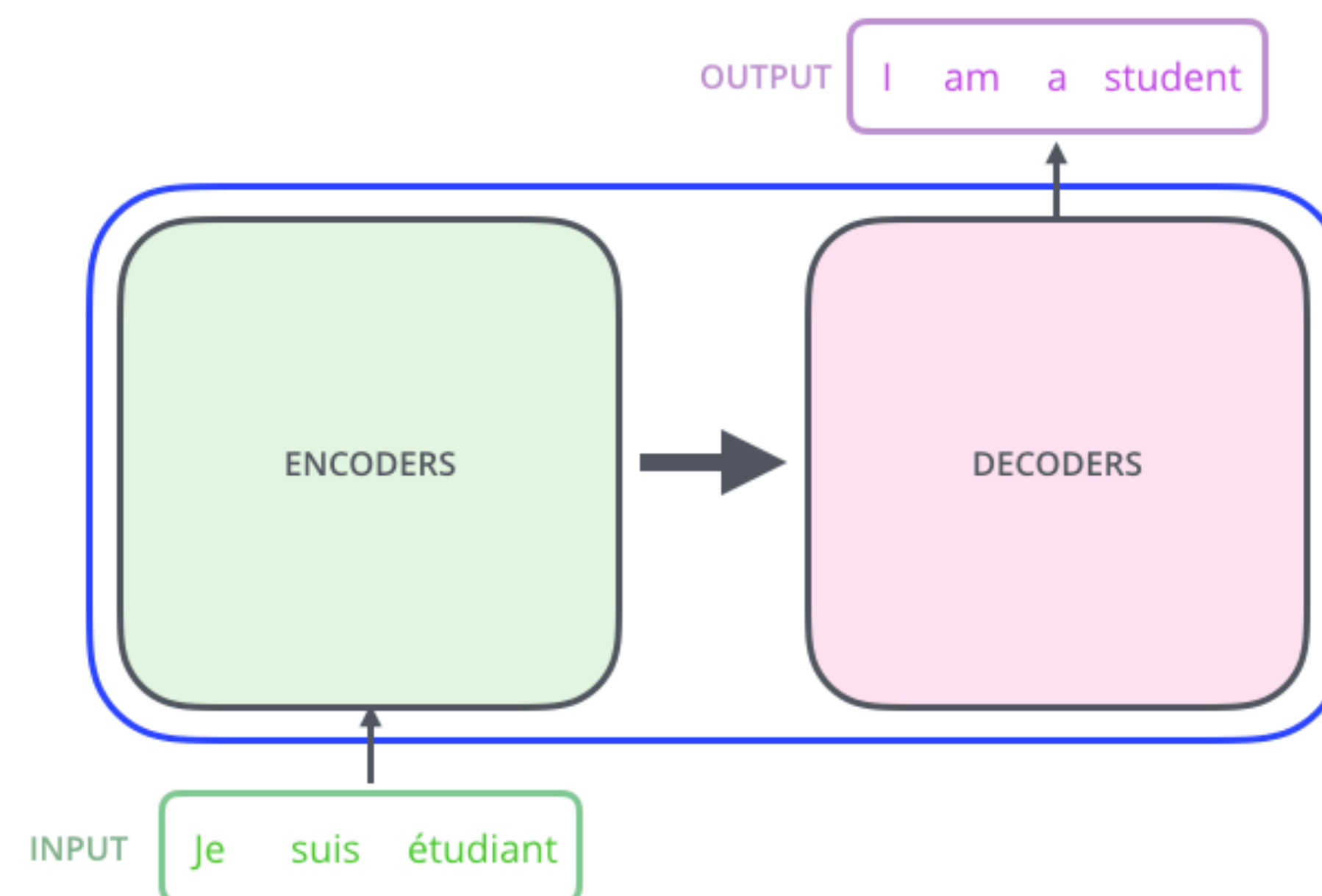


<https://jalammar.github.io/illustrated-transformer/>

Pre-trained Transformer

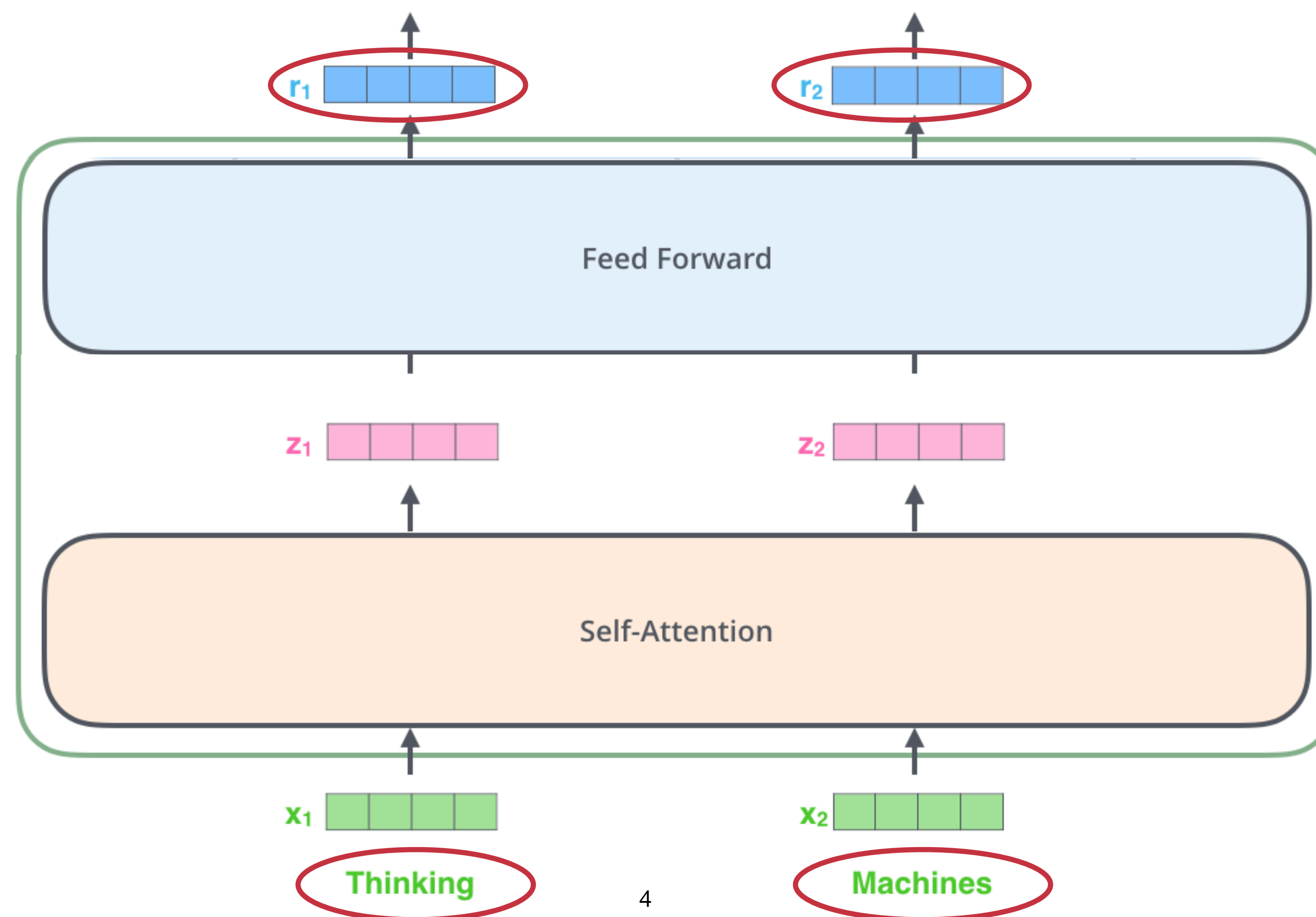
Если есть обученный для перевода Transformer, то:

- **Encoder:** выучил хорошие признаки для слов на языке входа
- **Decoder:** выучил хорошие признаки для слов на языке выхода



Pre-trained Transformer

Для каждого слова Transformer block выдает эмбединг, зависящий от всего контекста



Pre-trained Transformer

Inference:

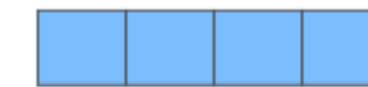
A plane crash



Transformer



*“plane” embedding
(in context 1)*



A plane surface



Transformer



*“plane” embedding
(in context 2)*



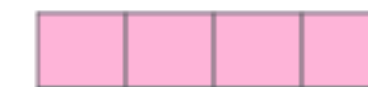
*A plane crash /
A plane surface*



Word2Vec



*“plane” embedding
(equal for both contexts)*

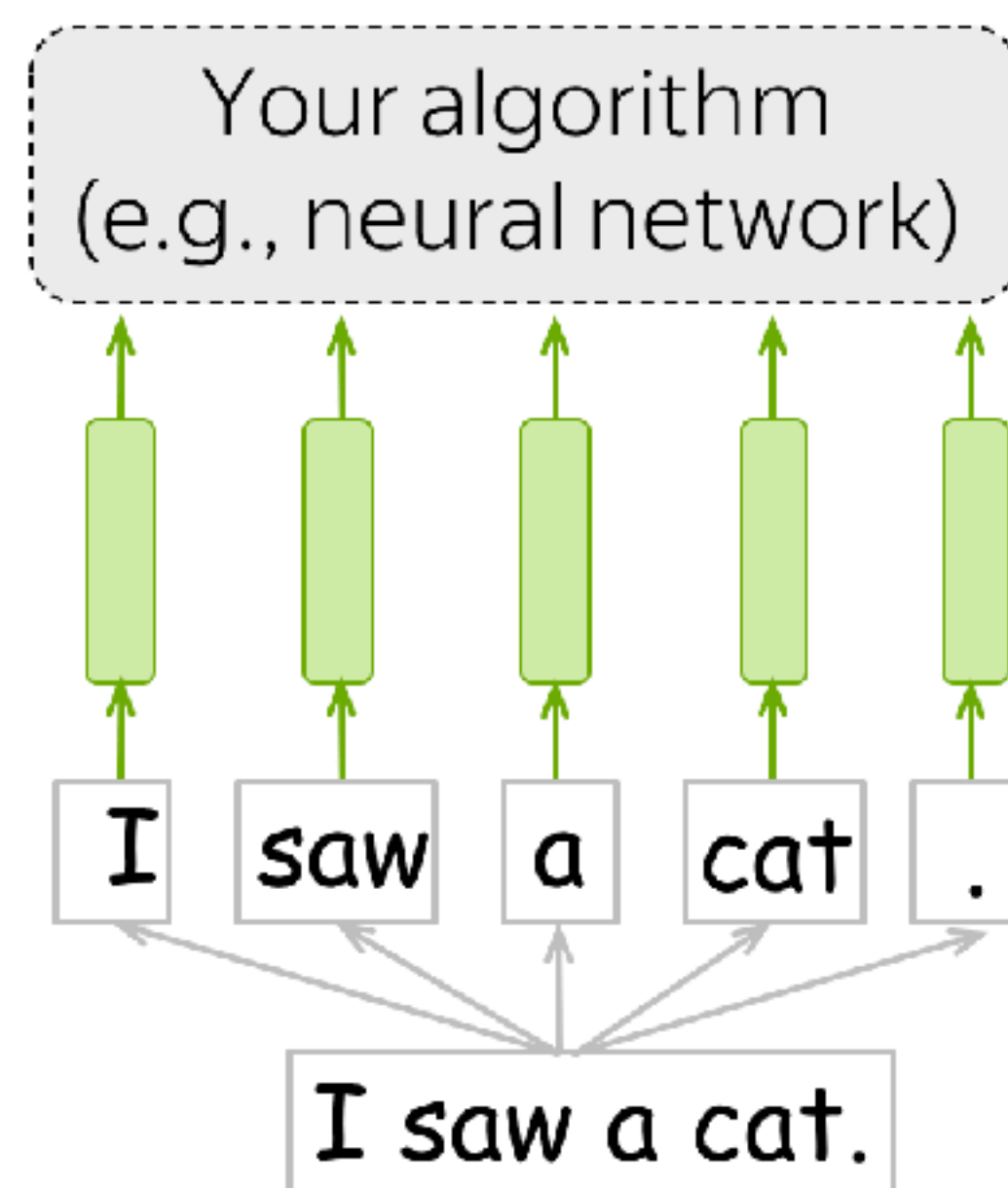


Pre-trained Transformer

Основная идея: взять предобученные эмбединги и дообучить на нужную задачу

Pre-train + Fine-tune (Transfer Learning)

Transfer learning (перенос обучения) — это подход в машинном обучении, при котором модель, обученная на одной задаче, используется для решения другой, связанной задачи.



Any algorithm for solving a task

/

Word representation - vector
(input for your model/algorithm)

Sequence of tokens

Text (your input)

Fine-tuning (дообучение) — это процесс адаптации предварительно обученной модели к новой, часто более специфичной задаче или набору данных, разновидность transfer learning. Как работает: получаем предобученную модель, адаптируем (fine-tune) с помощью небольшого набора данных, который имеет отношение к целевой задаче. В процессе fine-tuning можно: замораживать некоторые слои модели, оставляя их веса фиксированными, и обучать только верхние слои, которые отвечают за специфику новой задачи, дообучать все слои модели, чтобы она могла адаптировать свои представления под специфическую задачу. Это требует больше данных и вычислительных ресурсов.

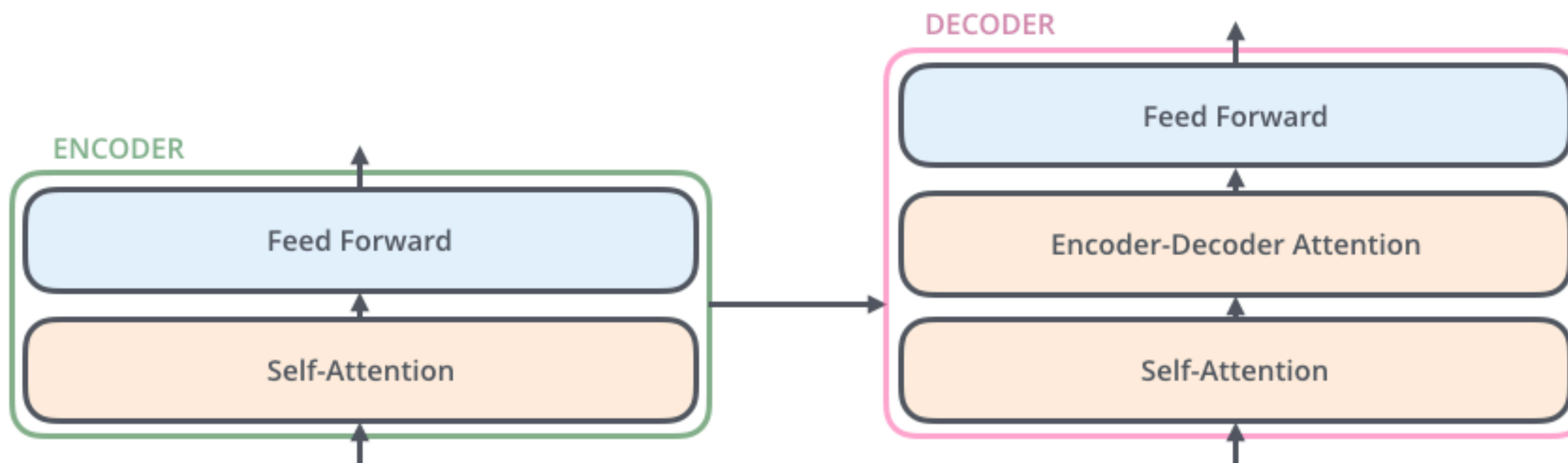
Image Credit

GPT

GPT

Generative Pre-Training

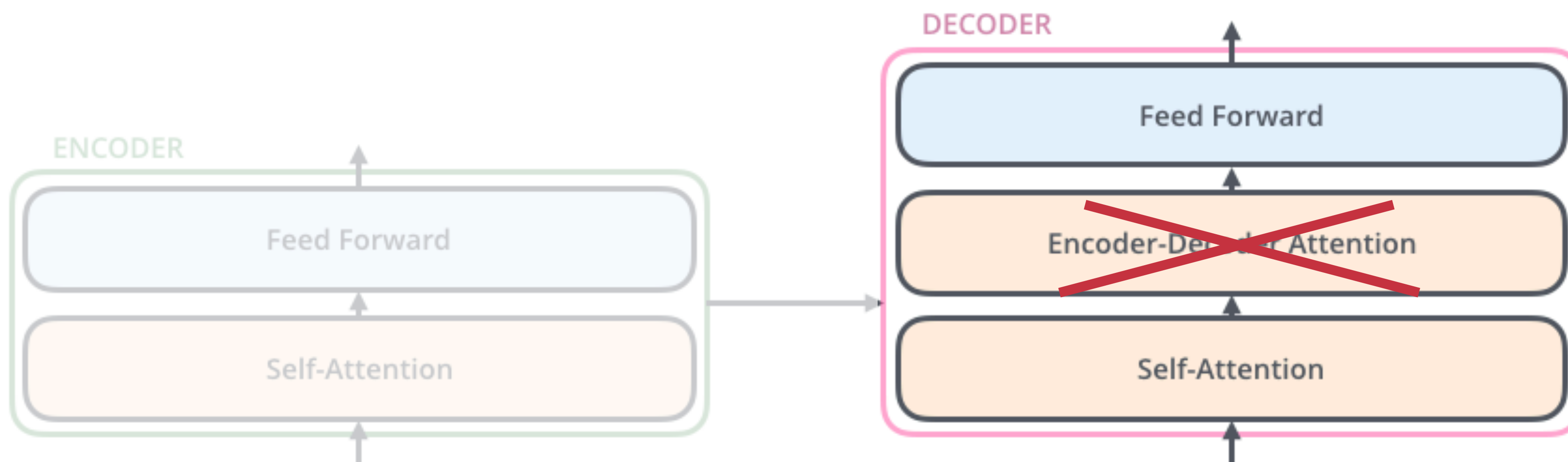
Архитектура: Transformer Decoder



GPT

Generative Pre-Training

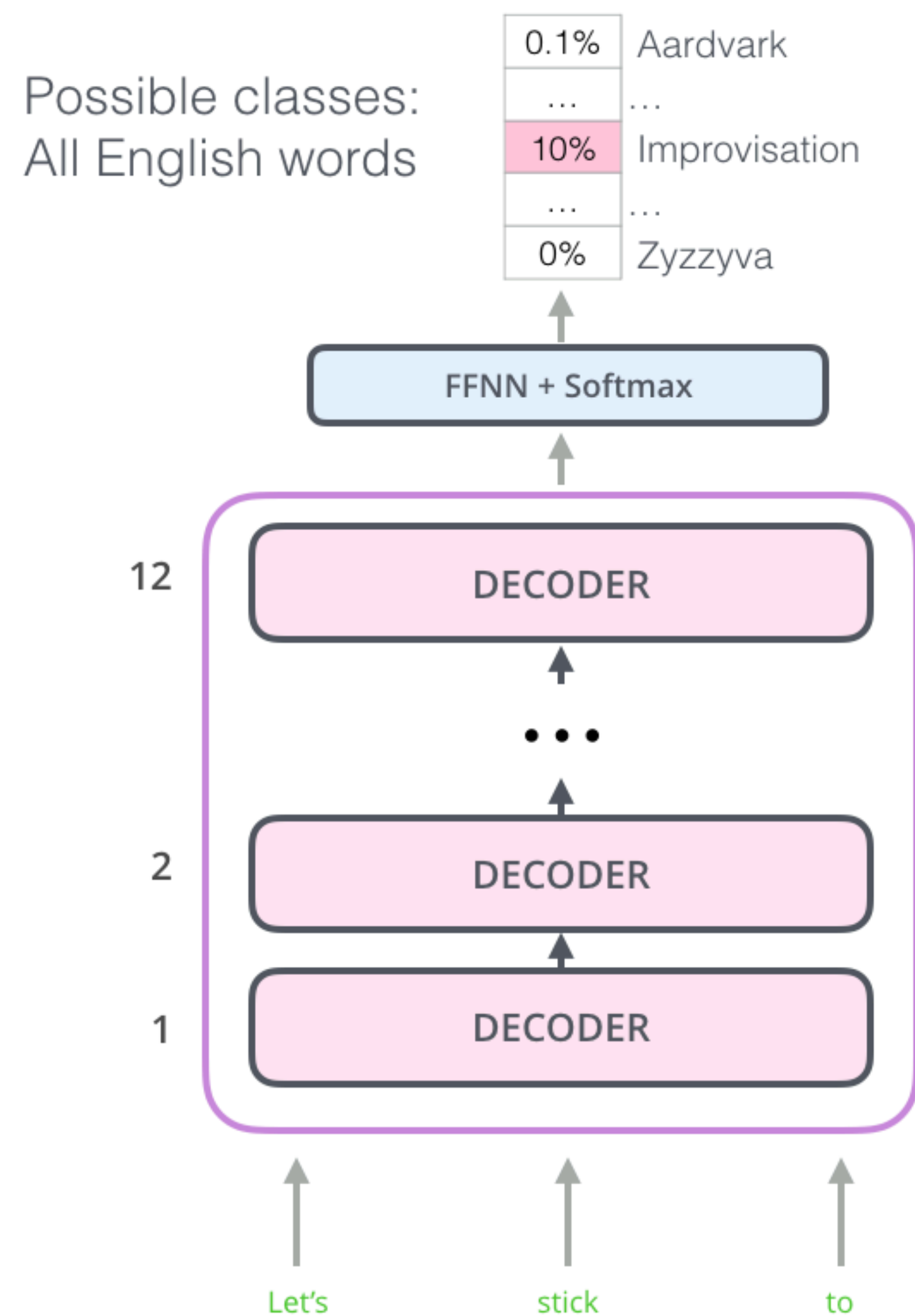
Архитектура: Transformer Decoder



GPT

Generative Pre-Training

Архитектура: Transformer Decoder



GPT

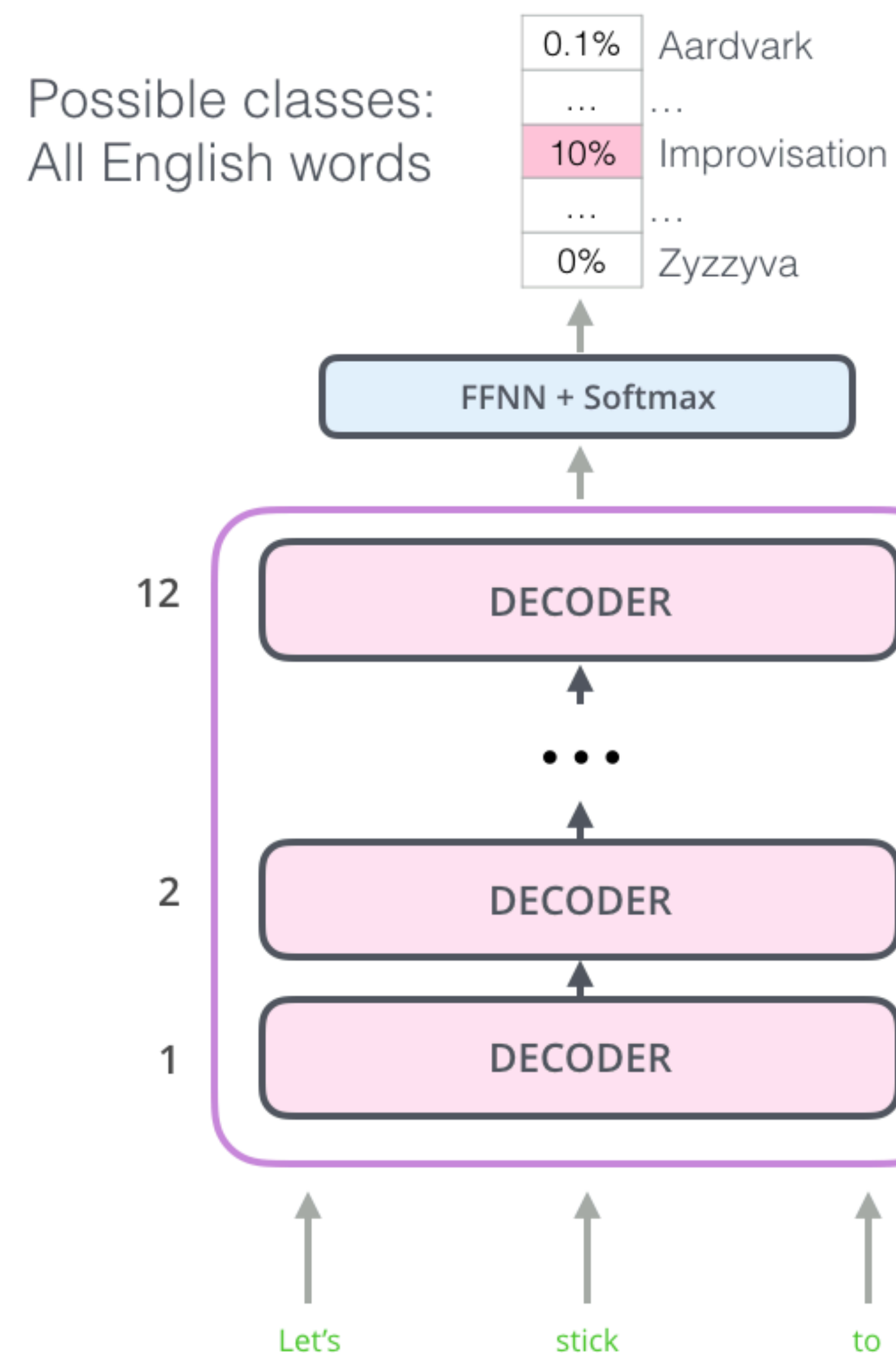
Generative Pre-Training

Архитектура: Transformer Decoder

Данные: тексты книг (BookCorpus)

+ Разнообразие

+ Большой объем



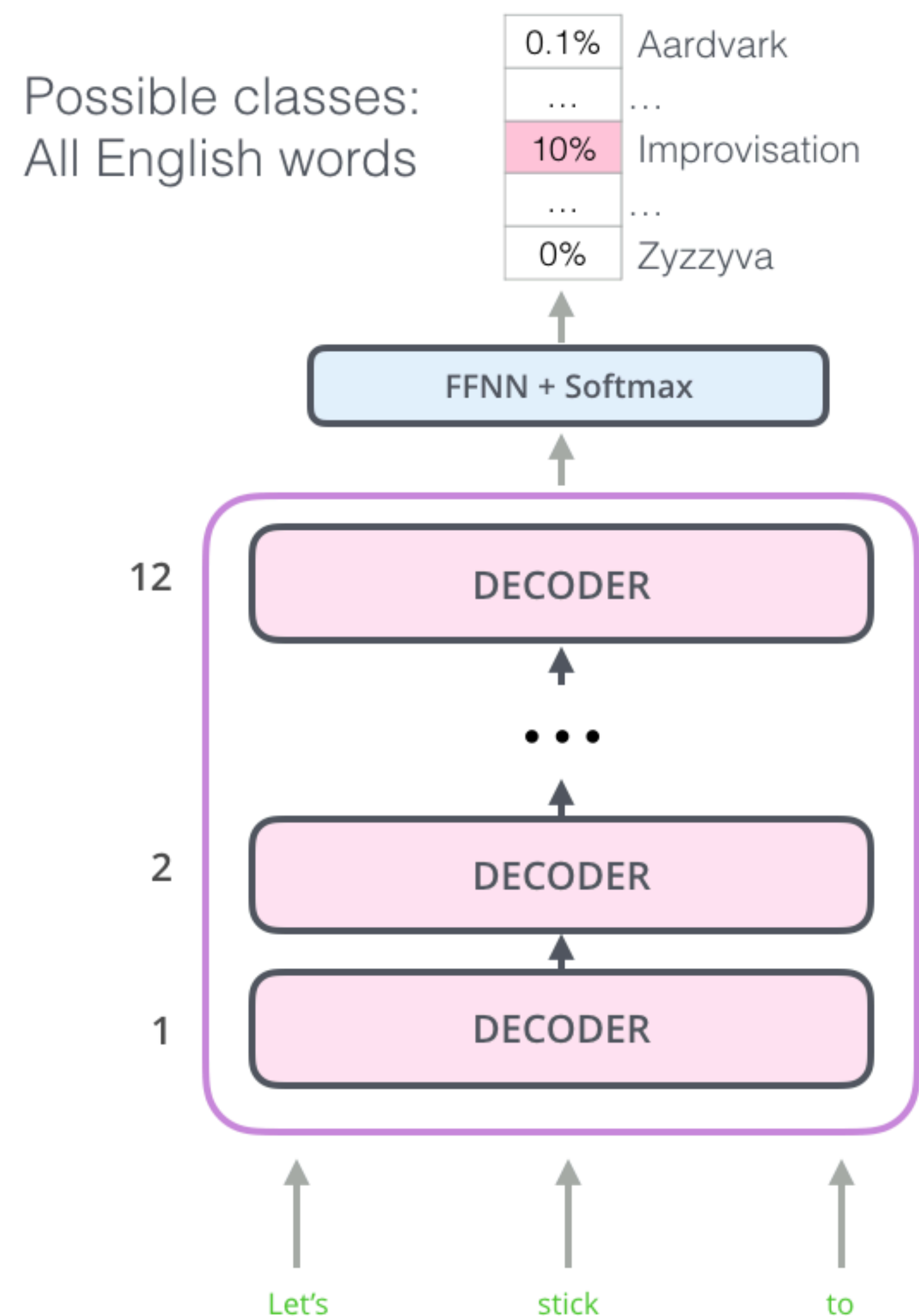
GPT

Generative Pre-Training

Архитектура: Transformer Decoder

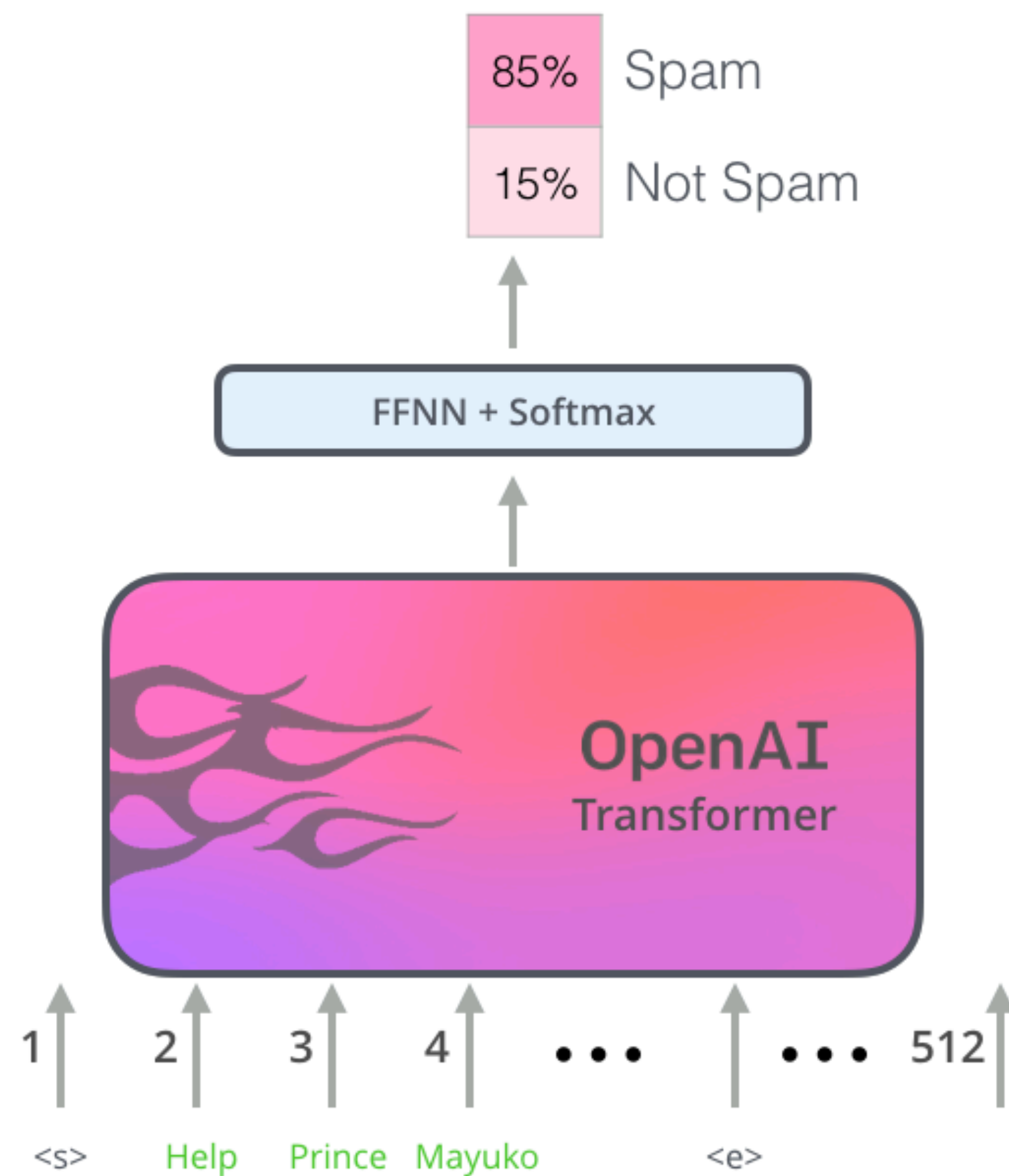
Данные: тексты книг (BookCorpus)

Задача для обучения: Language Modeling



GPT

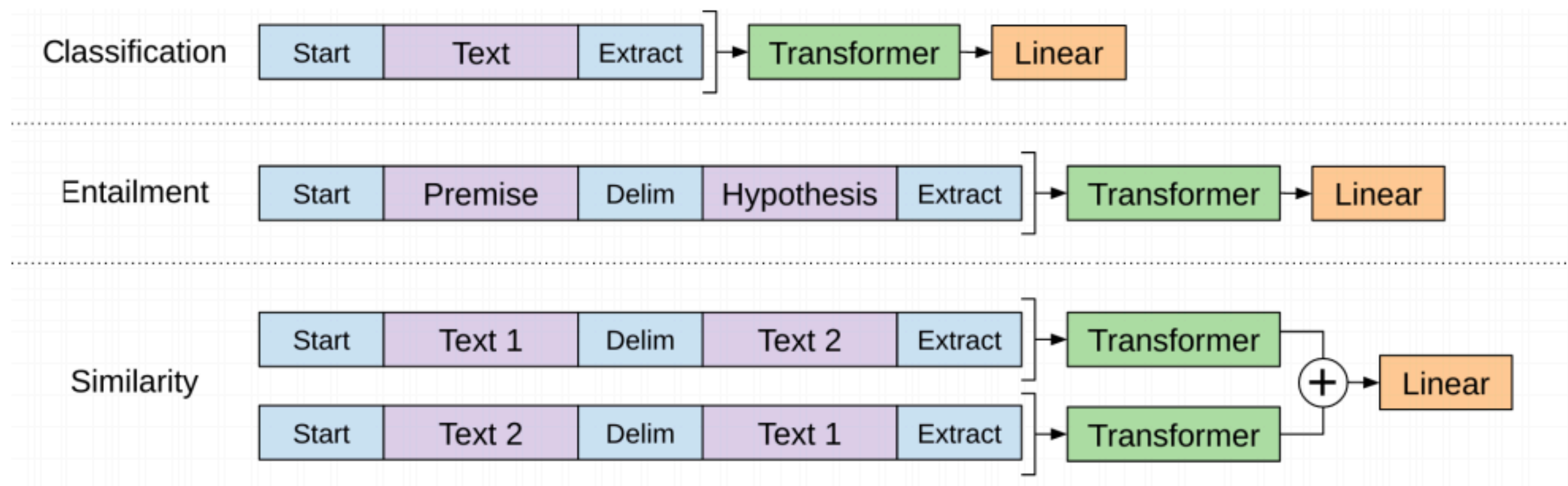
Как использовать?



GPT

Как использовать?

Для разных задач - разный формат входа



GPT-2

x10 параметров

x10 данных

GPT-2

Как использовать? **zero-shot task transfer**

Нет дообучения на новую задачу (fine-tuning)

Форматируем input, чтобы была понятна задача

Zero-shot task transfer — это подход в машинном обучении, при котором модель обучена выполнять задачу без явного обучения на примерах этой задачи

В рамках zero-shot подхода вы можете просто предоставить модели описание задачи, например: "Классифицируй следующий текст в одну из категорий: политика, спорт, наука, искусство." И, например, передать сам текст: "Сегодня прошел важный матч в Лиге чемпионов, где победу одержал клуб из Барселоны."

GPT-2

Форматируем input, чтобы была понятна задача

Пример: задача суммаризации

Article: Amina Ali Qassim is sitting with her youngest grandchild on her lap, wiping away tears with her headscarf. Only a few months old, this is the baby girl whose ears she desperately tried to cover the night the aerial bombardment started. She lay awake, she says, in a village mosque on the Yemeni island of Birim, counting explosions as the baby cried.

It could have been worse though. They could have still been in their house when the first missile landed.

"Our neighbor shouted to my husband 'you have to leave, they're coming.' And we just ran. As soon as we left the house, the first missile fell right by it and then a second on it. It burned everything to the ground," Qassim tells us ...

TL;DR:

input

GPT-2

Форматируем input, чтобы была понятна задача

Пример: задача суммаризации

Article: Amina Ali Qassim is sitting with her youngest grandchild on her lap, wiping away tears with her headscarf. Only a few months old, this is the baby girl whose ears she desperately tried to cover the night the aerial bombardment started. She lay awake, she says, in a village mosque on the Yemeni island of Birim, counting explosions as the baby cried.

It could have been worse though. They could have still been in their house when the first missile landed.

"Our neighbor shouted to my husband 'you have to leave, they're coming.' And we just ran. As soon as we left the house, the first missile fell right by it and then a second on it. It burned everything to the ground," Qassim tells us ...

TL;DR: Yemen is in the middle of a civil war. Saudi Arabia is leading the coalition bombing campaign. It's been bombing Yemen for more than two months now.



GPT-2 prediction

Input

GPT-2

Language Modeling (генерация текста)

Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.



GPT-3

x100 параметров (по сравнению с GPT-2)

x5 данных (по сравнению с GPT-2)

* нет в свободном доступе

GPT-3

x100 параметров (по сравнению с GPT-2)

x5 данных (по сравнению с GPT-2)

Как использовать? **zero-shot/one-shot/few-shot settings**

- показываем 0/1/несколько примеров

One-shot learning (обучение на одном примере) — это подход, при котором модель учится выполнять задачу, используя только один пример. Задача: Классификация текста. Пример 1: "Этот текст о футболе." Задача: "Определите, к какой категории относится следующий текст." Текст: "Сегодня прошел важный матч Лиги чемпионов." Ответ модели: "спорт".

В few-shot learning модель получает несколько примеров и на основе них обучается выполнять задачу. Этот подход использует методы обобщения и передачу знаний (transfer learning), чтобы эффективно работать с ограниченным количеством данных.

Пример: Задача: Классификация текста. Примеры: "Текст о футболе." → категория: "спорт" "Текст о политике." → категория: "политика" Задача: "Классифицируйте следующий текст." Текст: "Сегодня прошел важный матч Лиги чемпионов." Ответ модели: "спорт"

GPT-3

х100 параметров (по сравнению с GPT-2)

х5 данных (по сравнению с GPT-2)

Как использовать? **zero-shot/one-shot/few-shot settings**

- показываем 0/1/несколько примеров

English sentence 1	=	France sentence 1	Input
English sentence 2	=	France sentence 2	
English sentence 3	=		

GPT-3 prediction

GPT-3

Prompt:

The following are ideas for startups, apps, marketplaces, platforms:

###

A startup that helps content creators, such as photographers and videographers, monetize their work.

###

An online forum that helps people find roommates

###

GPT-3:

A community hub for entrepreneurs to share ideas and ask questions of experienced entrepreneurs

###

A website that helps people discover new interests and hobbies.

###

A company that helps brands advertise their products to potential buyers on Instagram.

###

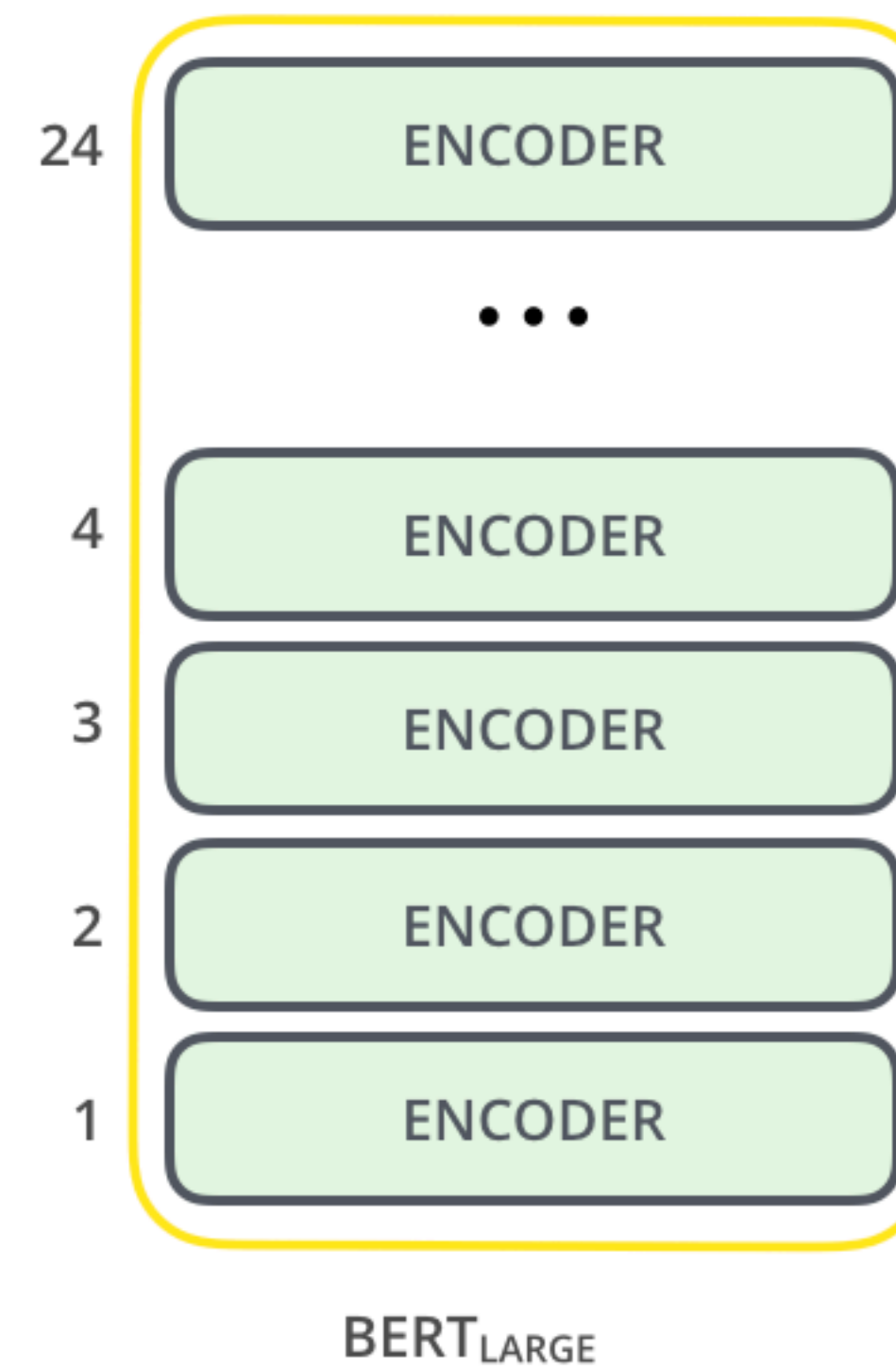
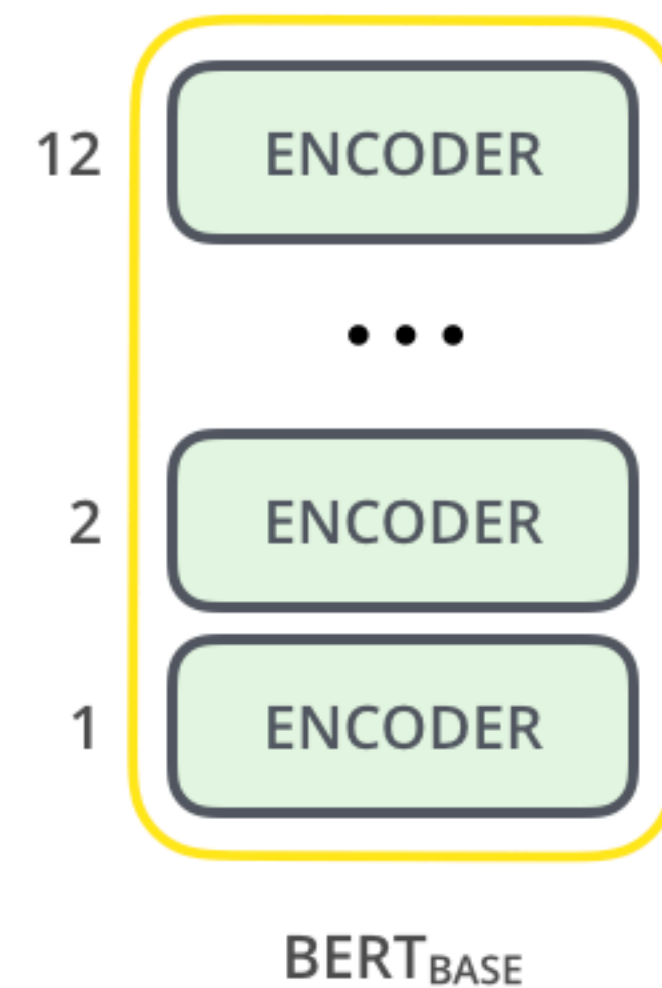


BERT

BERT

Bidirectional Encoder Representations from Transformers

Архитектура: Transformer Encoder



BERT

Bidirectional **E**ncoder **R**epresentations from **T**ransformers

Архитектура: Transformer Encoder

Данные: Wikipedia и тексты книг (BookCorpus)

BERT

Bidirectional **E**ncoder **R**epresentations from **T**ransformers

Архитектура: Transformer Encoder

Данные: Wikipedia и тексты книг (BookCorpus)

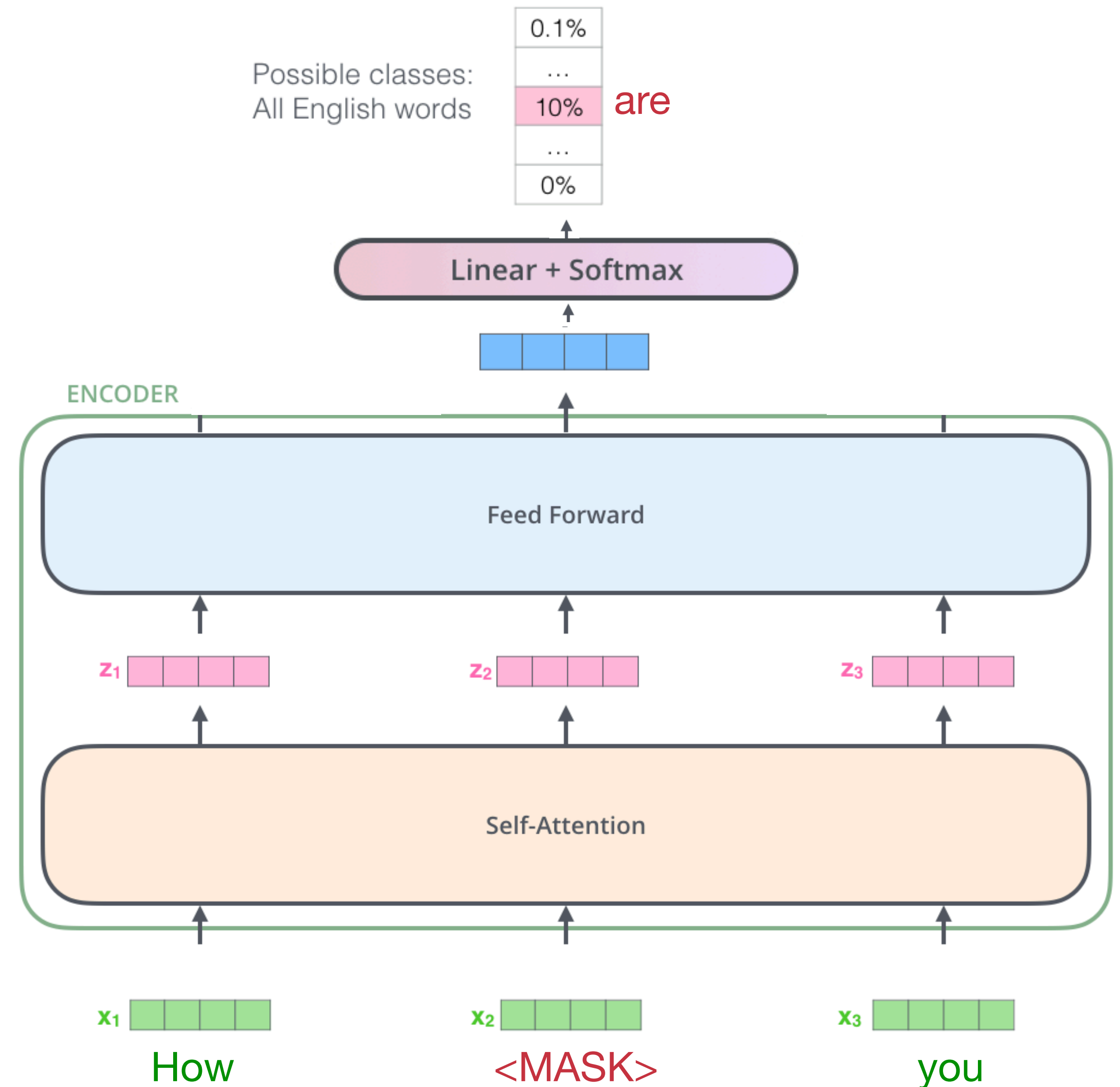
Задачи для обучения: Masked LM и Next Sentence Prediction

BERT

Masked Language Model

Случайно выбираем 15% позиций и заменяем на <MASK>

Задача: предсказать исходный токен



Приложение: Маскирование в декодерах

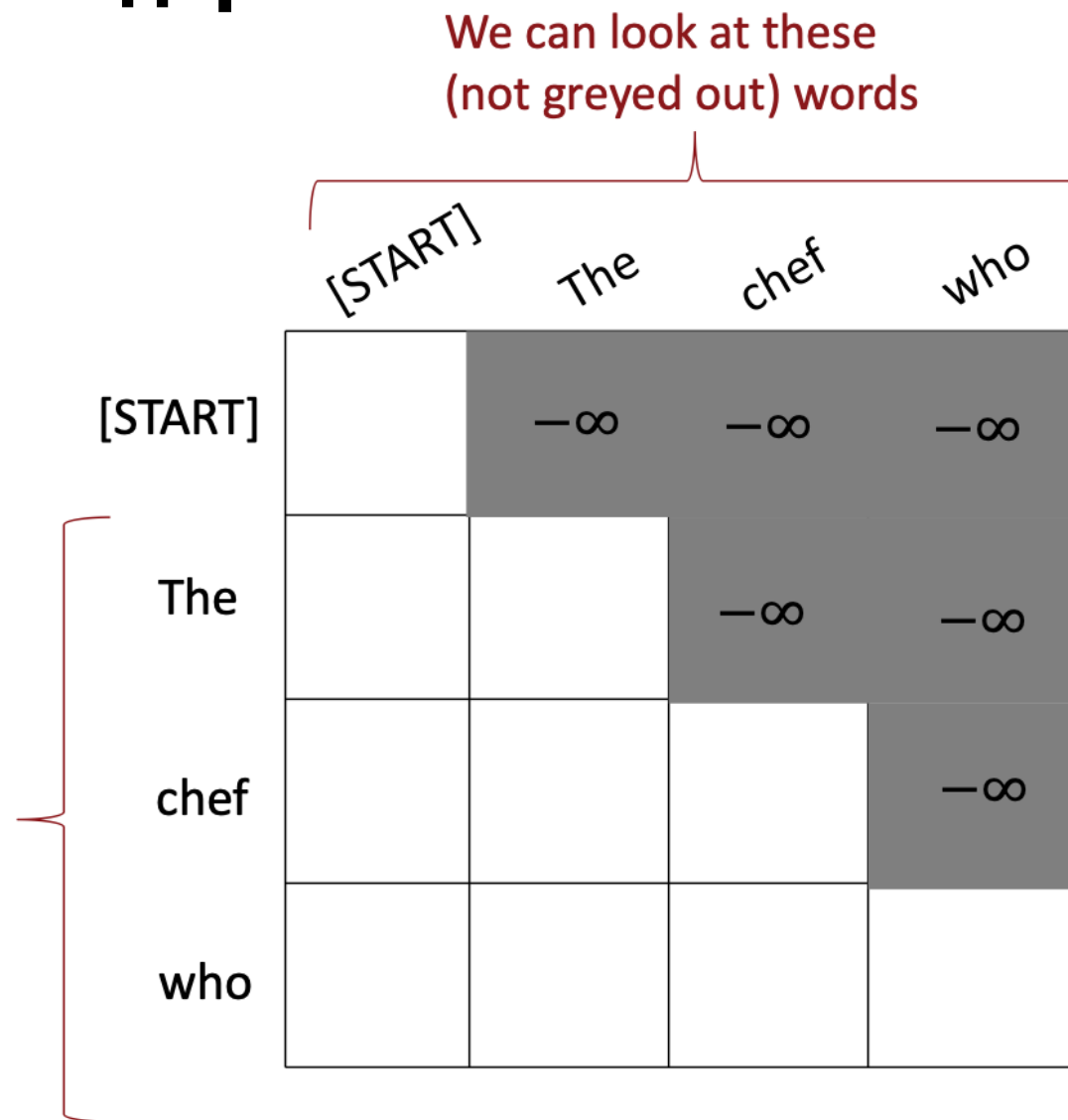
Маскирование в декодерах отличается от обучения с [MASK] токенами в энкодерах

В декодерах перекрывается всё, что расположено левее рассматриваемого токена, а значение скалярного произведения закладывается как $-\infty$

- To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$.

$$e_{ij} = \begin{cases} q_i^\top k_j, & j \leq i \\ -\infty, & j > i \end{cases}$$

For encoding these words



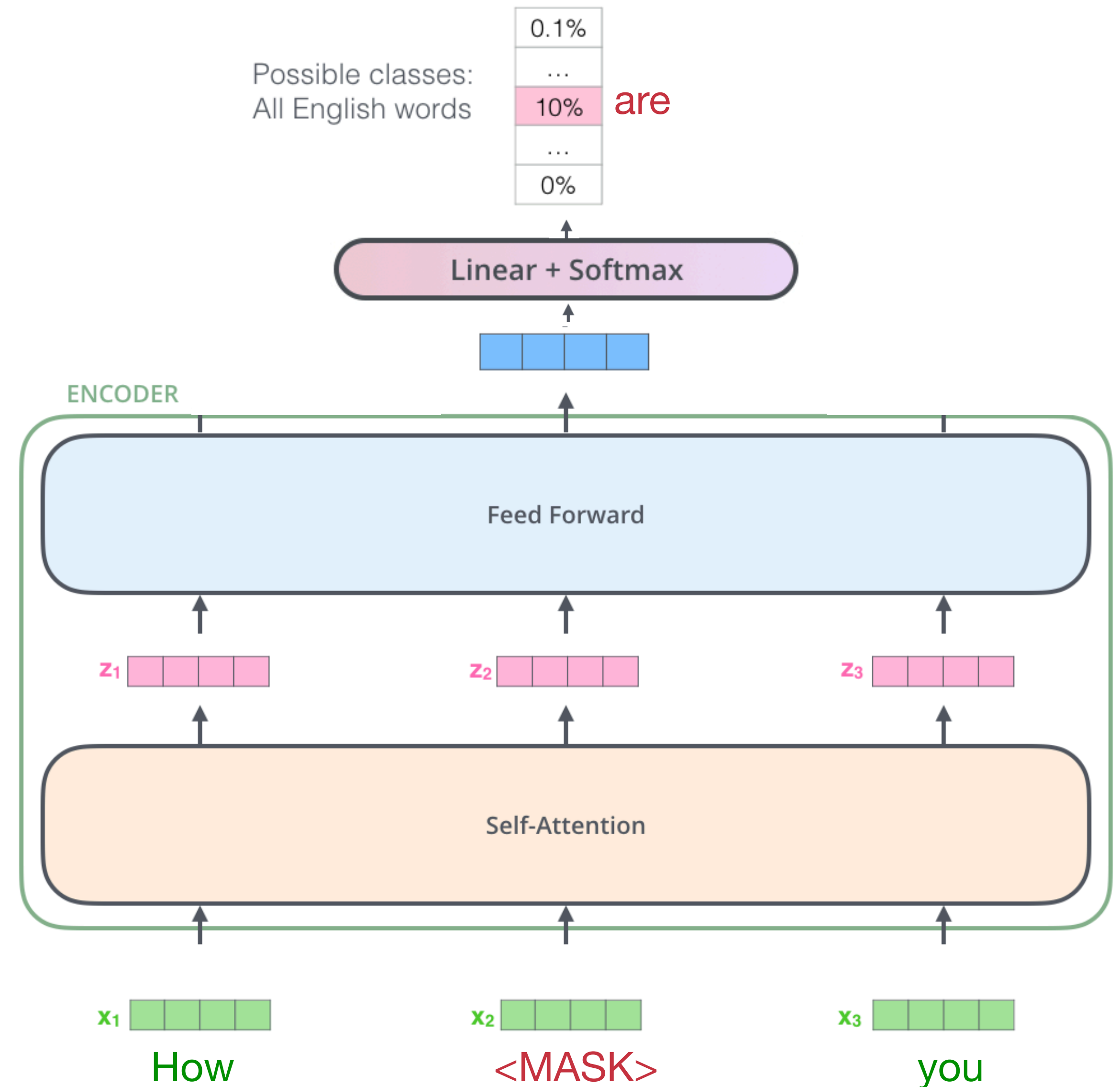
BERT

Masked Language Model

Случайно выбираем 15% позиций:

- 80% заменяем на <MASK>
- 10% заменяем на случайный токен
- 10% оставляем

Задача: предсказать исходный токен



BERT

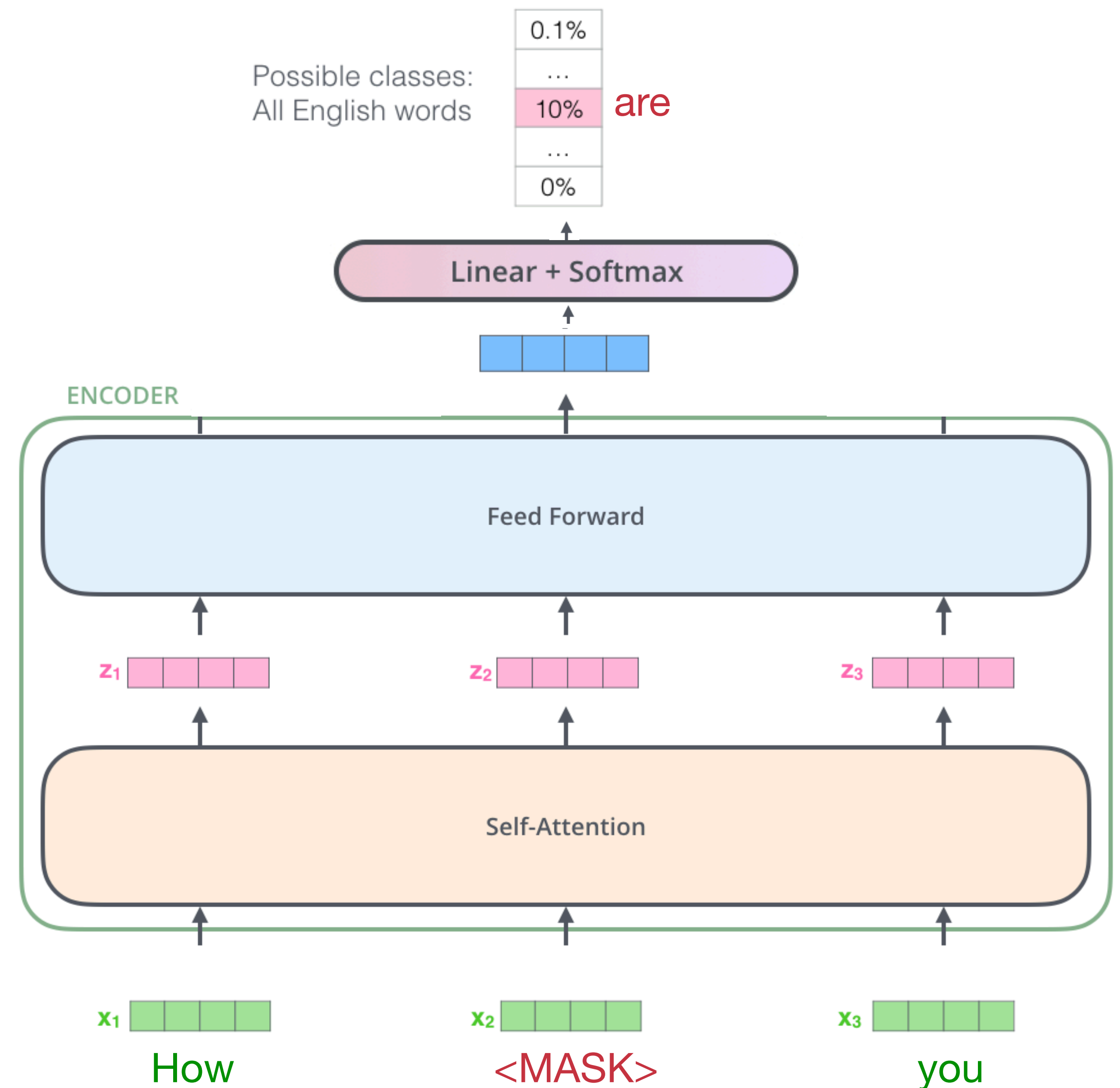
Masked Language Model

Случайно выбираем 15% позиций:

- 80% заменяем на <MASK>
- 10% заменяем на случайный токен
- 10% оставляем

Задача: предсказать исходный токен

Обученные эмбеддинги учитывают
контекст слева и справа



BERT

Next Sentence Prediction

Для некоторых задач нужно понимать взаимоотношения между двумя предложениями:

- Similarity
- Entailment
- Question Answering
- ...

BERT

Next Sentence Prediction

Вход: 2 предложения (A и B)

50% - B следует за A в тексте

50% - B выбрано случайно

Формат входа:

[CLS] the man [MASK] to the store [SEP] penguin [MASK] are flightless birds [SEP]

└────────────────────────────────┘ └────────────────────────────────┘

Sentence A Sentence B

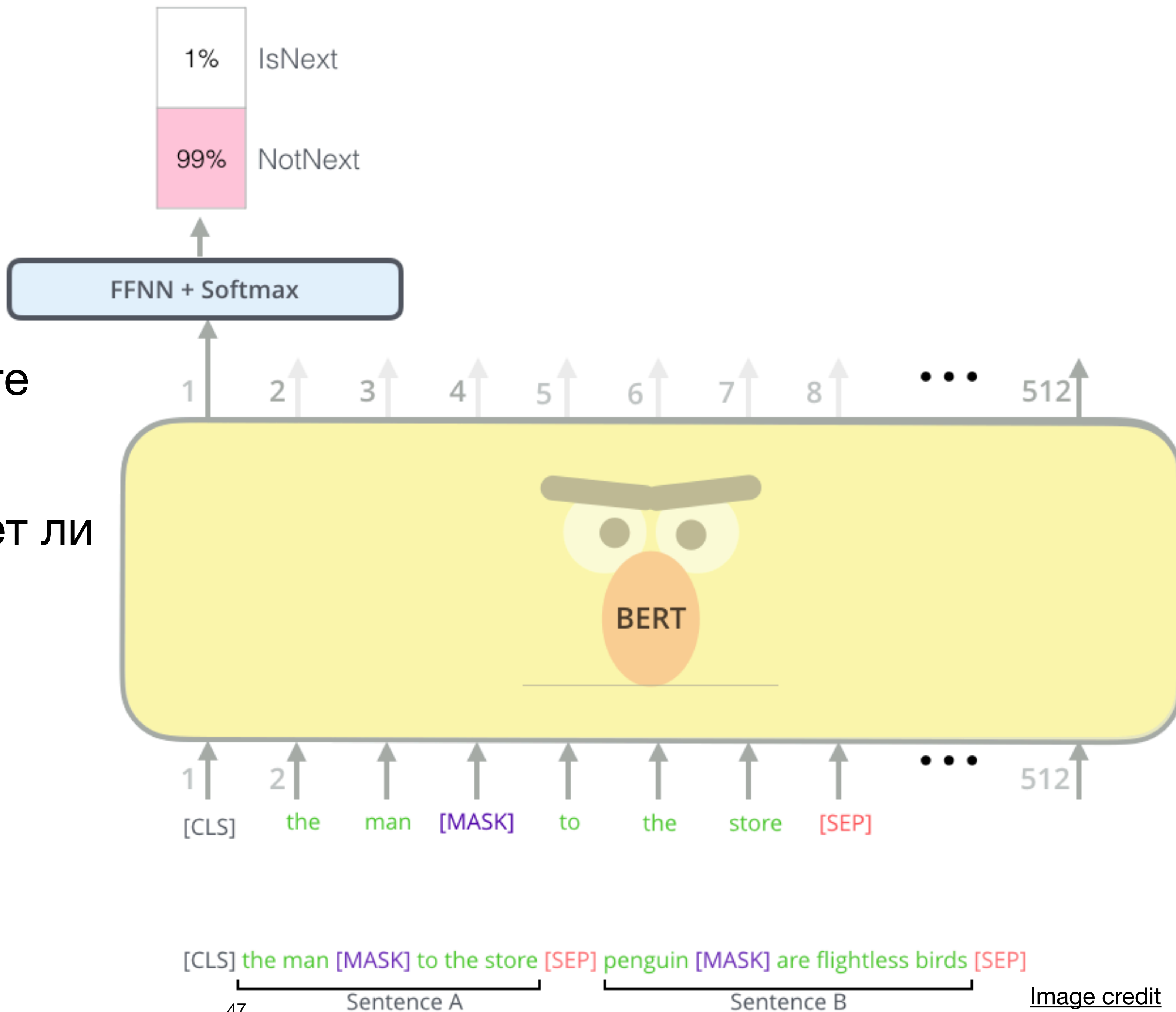
BERT

Next Sentence Prediction

Вход: 2 предложения (A и B)

50% - B следует за A в тексте
50% - B выбрано случайно

Задача: предсказать, следует ли
B за A



BERT

Next Sentence Prediction

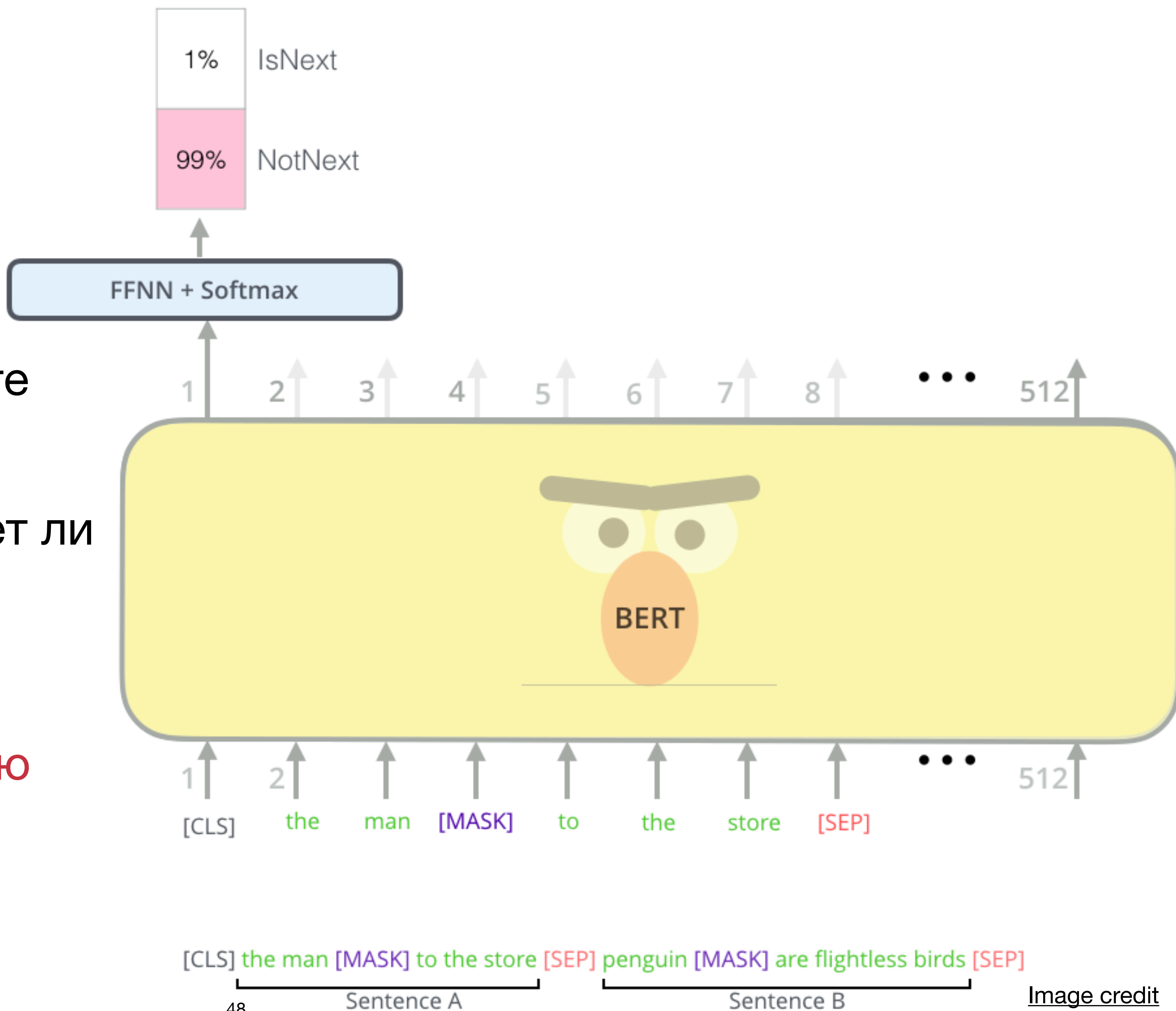
Вход: 2 предложения (A и B)

50% - B следует за A в тексте
50% - B выбрано случайно

Задача: предсказать, следует ли
B за A

<CLS> - выучивает
агрегированную информацию

<SEP> - разделитель



BERT

Как использовать?

- Linear+Softmax поверх <CLS> - для задач классификации предложения (или двух)
- Linear+Softmax поверх всех выходов - для задач классификации токенов
- Выходы BERT - как вход в другие модели (task-specific)
- ...

RoBERTa

Robustly Optimized **BERT** Pretraining Approach

Улучшенная версия BERT:

- x10 данных
- x10 вычислительных ресурсов (дольше обучение, больше batch size)
- Не использовали задачу Next Sentence Prediction

Улучшение в 2-20% (в зависимости от задачи)

BART and T5

BART

Bidirectional and Auto-Regressive Transformers

Идея: соединить преимущества BERT и GPT

Архитектура: Transformer Encoder-Decoder

Данные: как в RoBERTa

Гибридная архитектура: BART сочетает двухкомпонентную модель трансформера (энкодер и декодер), что позволяет использовать преимущества как двунаправленного внимания (подобного BERT), так и авторегрессивной генерации (подобной GPT).

В отличие от T5, который использует текст как вход и выход, BART обучается на задачах восстановления текста, где входной текст и его зашумленная версия используются для обучения:

Пример: Входной текст может быть: "Этот фильм был восхитительным", а зашумленная версия: "Этот фильм был ____". Модель должна восстанавливать пропущенное слово.

BART

Bidirectional and Auto-Regressive Transformers

Идея: соединить преимущества BERT и GPT

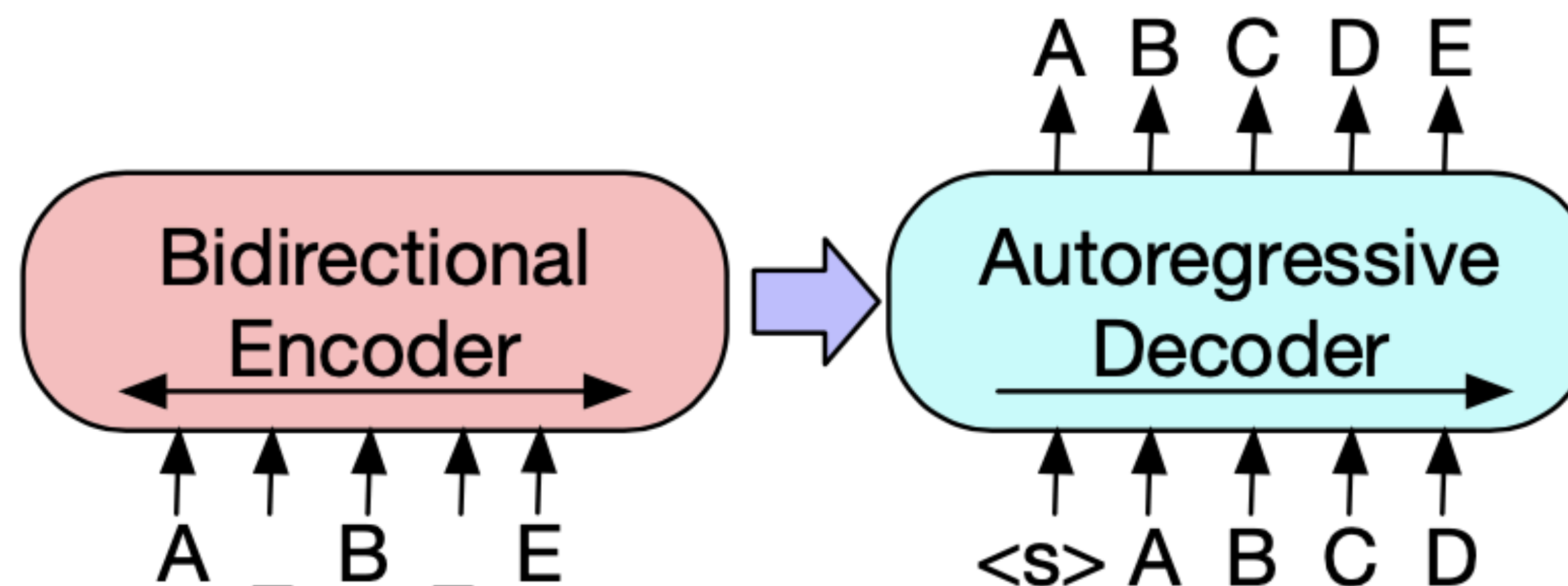
Архитектура: Transformer Encoder-Decoder

Данные: как в RoBERTa

Задача для обучения: восстановить последовательность

Последовательность: ABCDE

- Маскируем C, D
- Шум: лишняя маска перед B



T5

Text-to-Text Transfer Transformer

Идея: соединить преимущества BERT и GPT

Архитектура: Transformer Encoder-Decoder

Данные: x5 от данных RoBERTa

Задача для обучения: ВОССТАНОВИТЬ ПОСЛЕДОВАТЕЛЬНОСТЬ

T5 — это модель от Google Research, которая использует архитектуру трансформера, адаптированную для работы с текстом в формате "от текста к тексту". Все задачи, такие как перевод, суммирование, классификация, извлечение информации, преобразуются в задачу текстового преобразования. Таким образом, вход и выход всегда представлены в виде текста, что делает модель очень универсальной.

Основные особенности T5:

Текст как вход и выход: T5 преобразует все задачи NLP в задачу, где и вход, и выход — это текст. Например:

Перевод: Вход может быть "Переведи этот текст на французский: 'I love programming.'" и выход "J'adore programmer."

Классификация: Вход может быть "Классифицируй текст: 'Этот фильм замечательный.'", а выход — "Позитивный".

Суммирование: Вход — длинный текст, выход — краткое изложение.

Подход "Text-to-Text": В отличие от моделей, обученных специально для одной задачи (например, классификации или перевода), T5 решает задачи в универсальной текстовой форме⁵⁴, что делает его подход гибким и универсальным.

T5

Text-to-Text Transfer Transformer

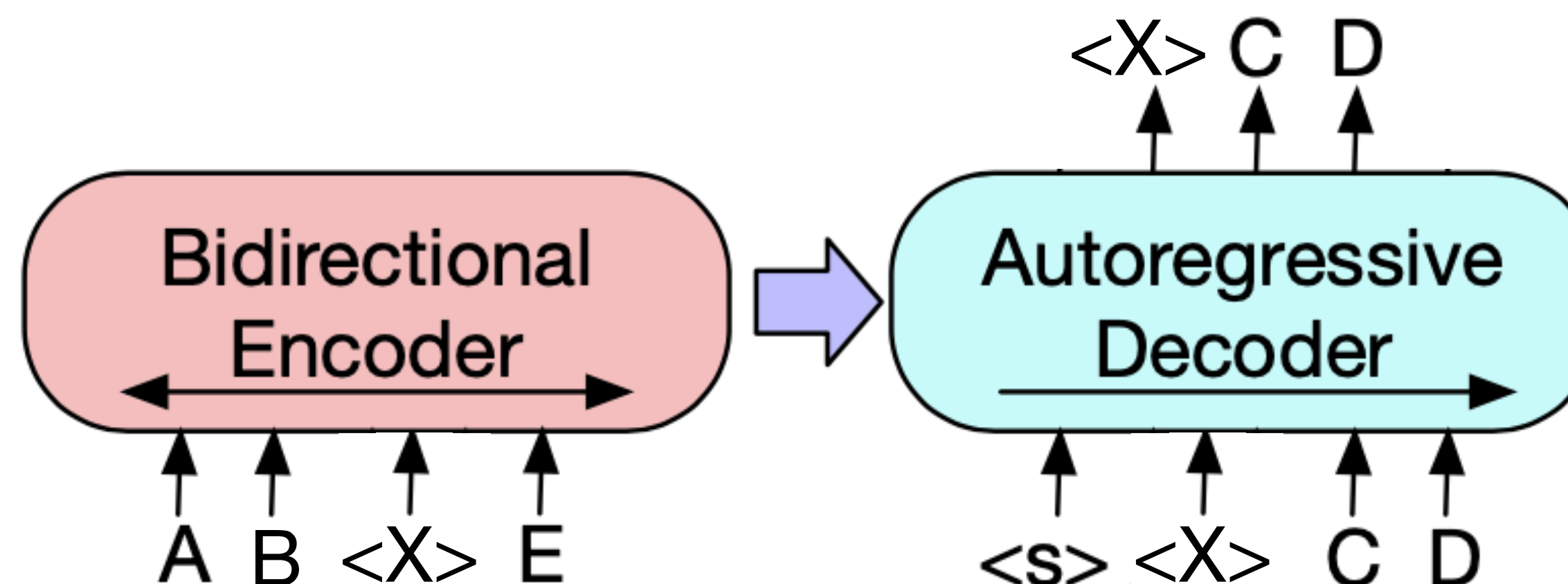
Идея: соединить преимущества BERT и GPT

Архитектура: Transformer Encoder-Decoder

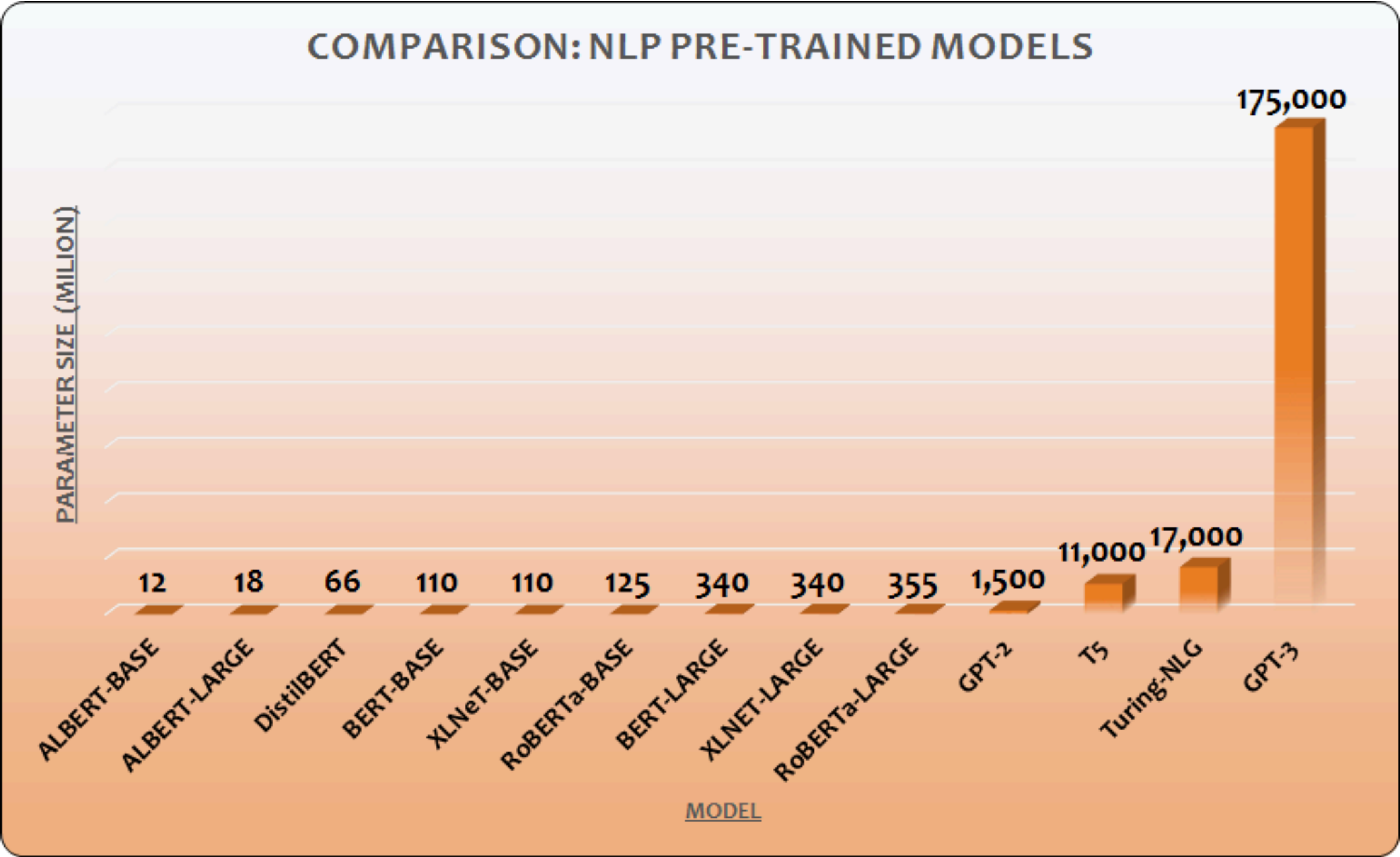
Данные: x5 от данных RoBERTa

Задача для обучения: ВОССТАНОВИТЬ ПОСЛЕДОВАТЕЛЬНОСТЬ

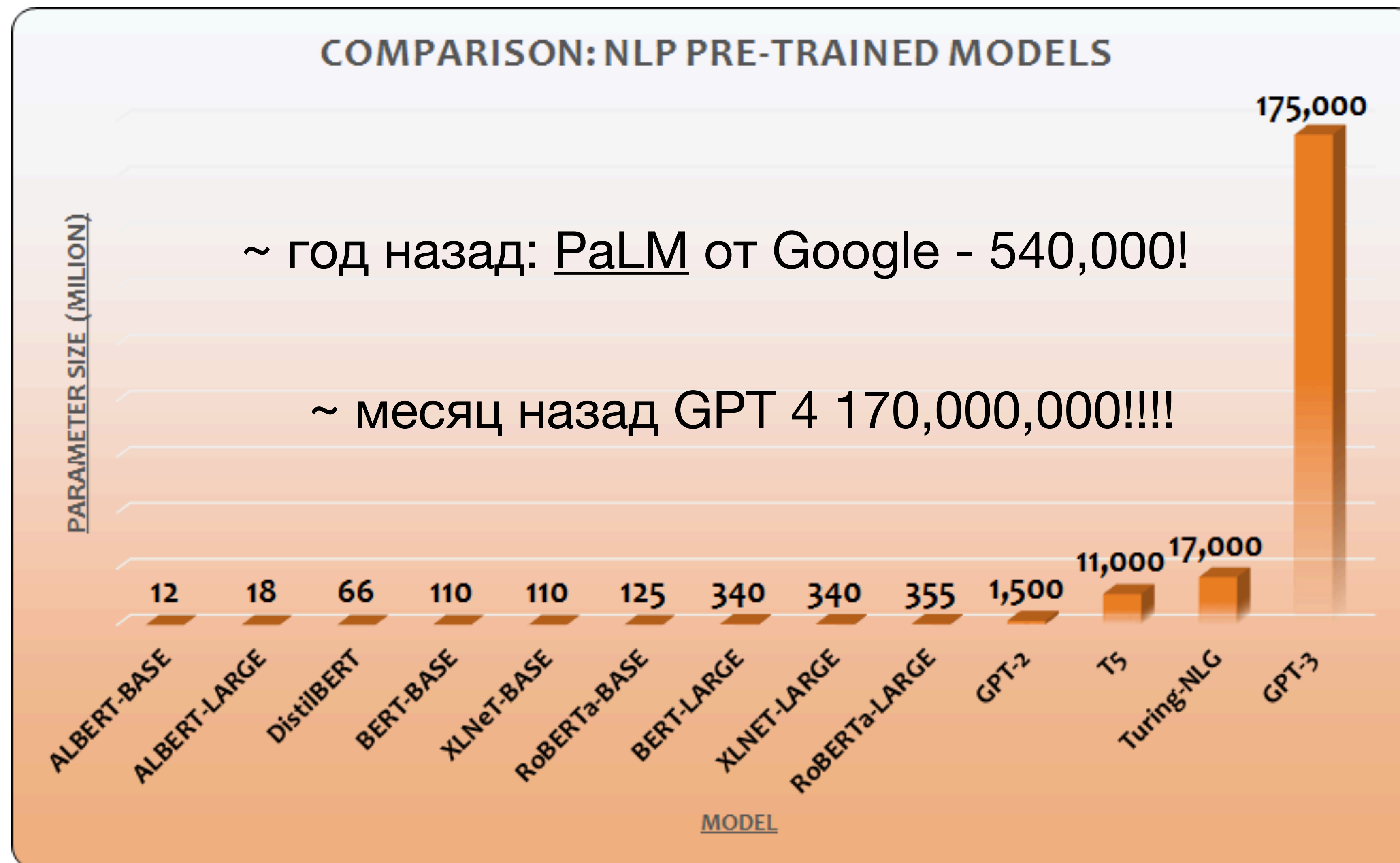
Вместо маски используем
специальные токены



Сравнение



Сравнение



BERT vs GPT vs ChatGPT

ChatGPT – разновидность GPT-3 моделей, дизайн которых создавался для chatbot приложения. Модель обучалась на большой датасете диалогов, так чтобы она могла генерировать ответы, подходящие для контекста мессенджеров, создавая связное обсуждение. При этом ChatGPT не такая мощная модель, как GPT-3, хотя и более быстрая

Bert и GPT-3 – различаются архитектурой, Bert – bidirectional model, GPT-3 – autoregressive.

GPT-models учитывают только контекст слева от рассматриваемого слова, каждый токен предсказывается и сопоставляется только с предыдущими токенами. Данная особенность делает GPT модели подходящими для генерации текстов, но хуже справляющимися с классификацией

Bert – рассматривает контекст в обе стороны. Чаще используется для задач классификации, генерирует весь output за раз. Основная задача Bert во время обучения – маскировать случайным образом слова и пытаться их предсказать по контексту справа и слева

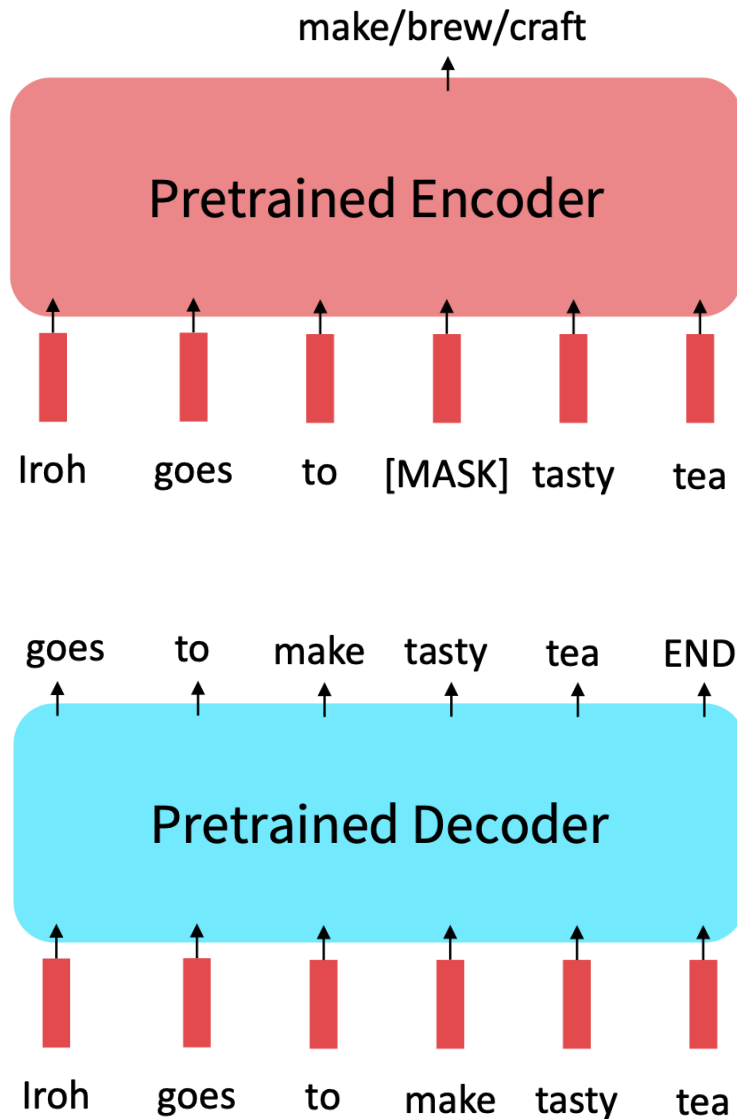
Bert

- **Encoder only**
- **Uses Self Attention**
- **Multiple models for different NLP tasks**
- **Fine Tuning on Custom Data**

GPT

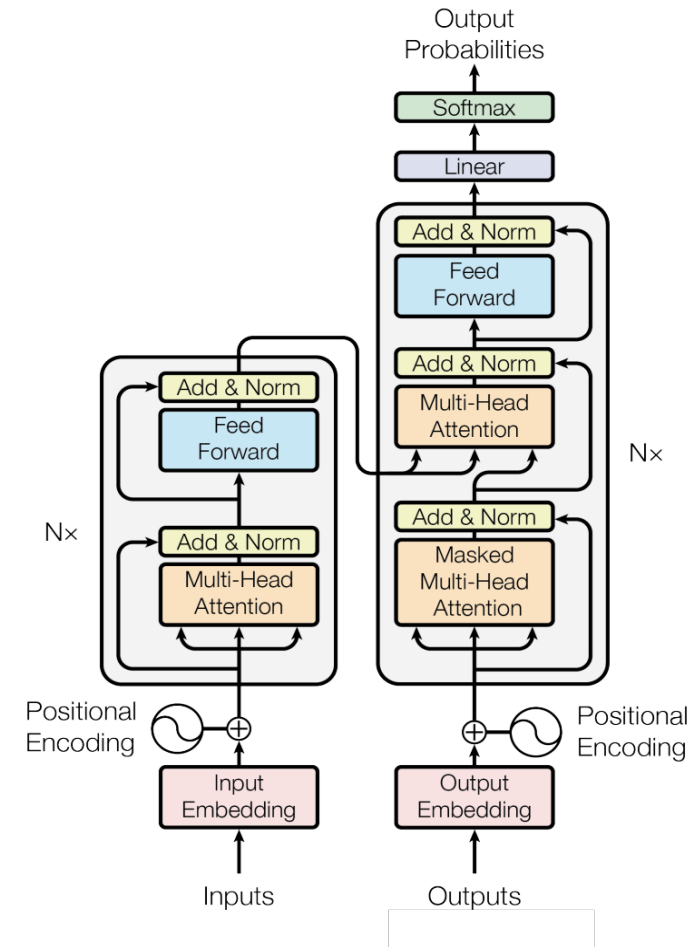
- **Decoder blocks only**
- **Uses Masked Self-Attention**
- **Single model for all NLP tasks**
- **Doesn't require fine tuning**

BERT vs GPT



BERT

Encoder



GPT

Decoder

1 - Semi-supervised training on large amounts of text (books, wikipedia..etc).

The model is trained on a certain task that enables it to grasp patterns in language. By the end of the training process, BERT has language-processing abilities capable of empowering many models we later need to build and train in a supervised way.

Semi-supervised Learning Step

Model:



Dataset:



Objective:

Predict the masked word
(language modeling)

2 - Supervised training on a specific task with a labeled dataset.

Supervised Learning Step

Model:
(pre-trained
in step #1)



Classifier

75% Spam
25% Not Spam

Dataset:

Email message	Class
Buy these pills	Spam
Win cash prizes	Spam
Dear Mr. Atreides, please find attached...	Not Spam

The two steps of how BERT is developed. You can download the model pre-trained in step 1 (trained on un-annotated data), and only worry about fine-tuning it for step 2. [Source for book icon].

Основные задачи, решаемые трансформерами

- General question answering, language understanding and classification (Bert)
- Text generation and understanding (Bart)
- Text generation, code generation, as well as image and audio generation (GPT-3)
- Object classification (CLIP)
- Dialog agent (ChatGPT)
- Text to image (GLIDE, DALL-E-2)
- Mathematical reasoning (Minerva)
- Summarization (Pegasus)
- Translation and other cross-lingual language tasks (XLM-RoBERTa)

Основные задачи, решаемые трансформерами 2

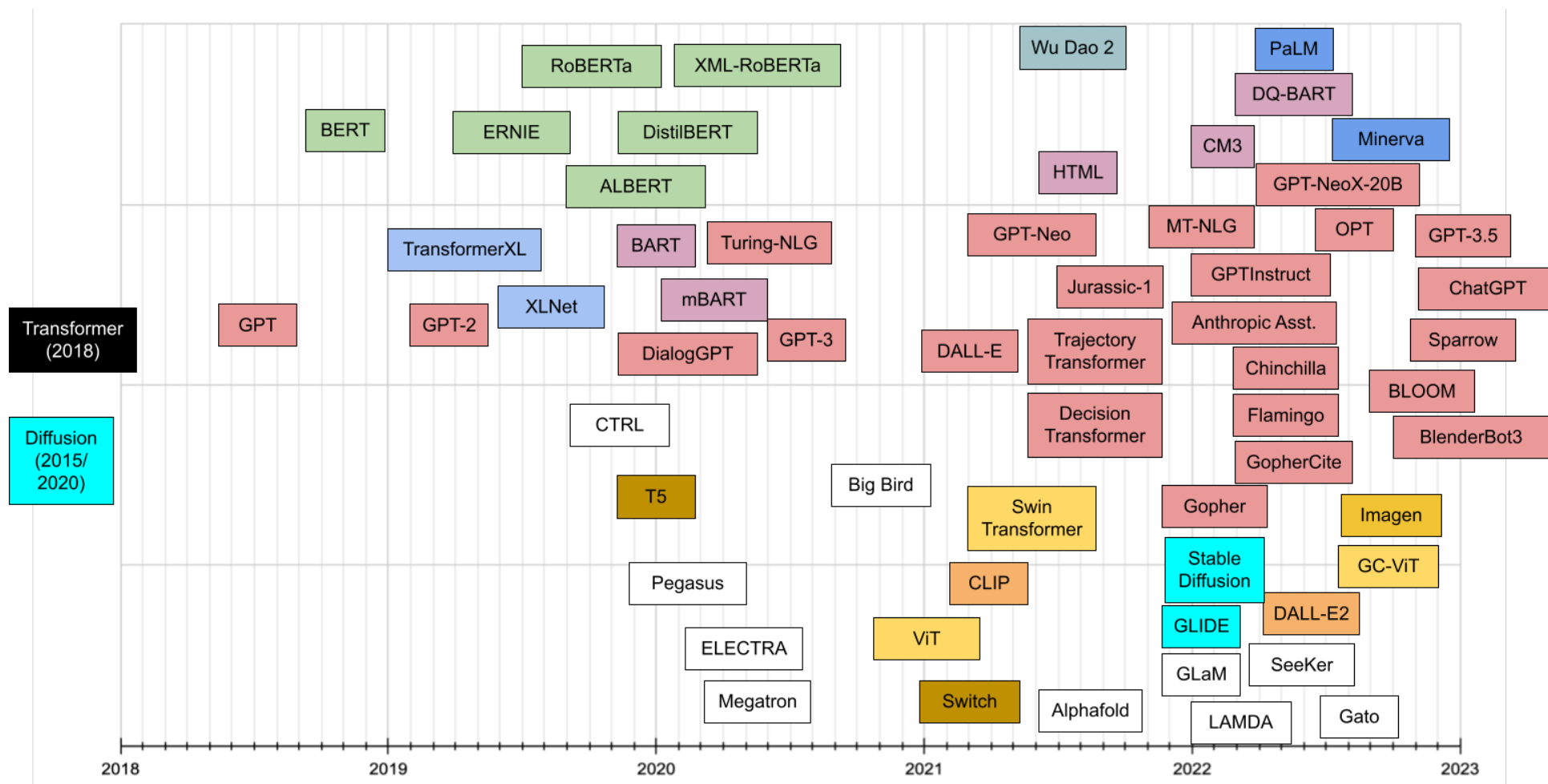
- Трансформеры применяются и для анализа табличных данных, например, в статье «Revisiting Deep Learning Models for Tabular Data»*, подготовленной командой Яндекса сравниваются различные модели (MLP, ResNet, FT-Transformer и т.д.) и предпочтение отдается трансформерам, однако результат зависит от специфики задачи, например, для Long Time Series forecasting существуют модели проще по структуре и превосходящие трансформеры: « Are Transformers Effective for Time Series Forecasting? »**. Кроме того, кол-во данных существенно влияет на выбор модели, так как увеличивается риск переобучения



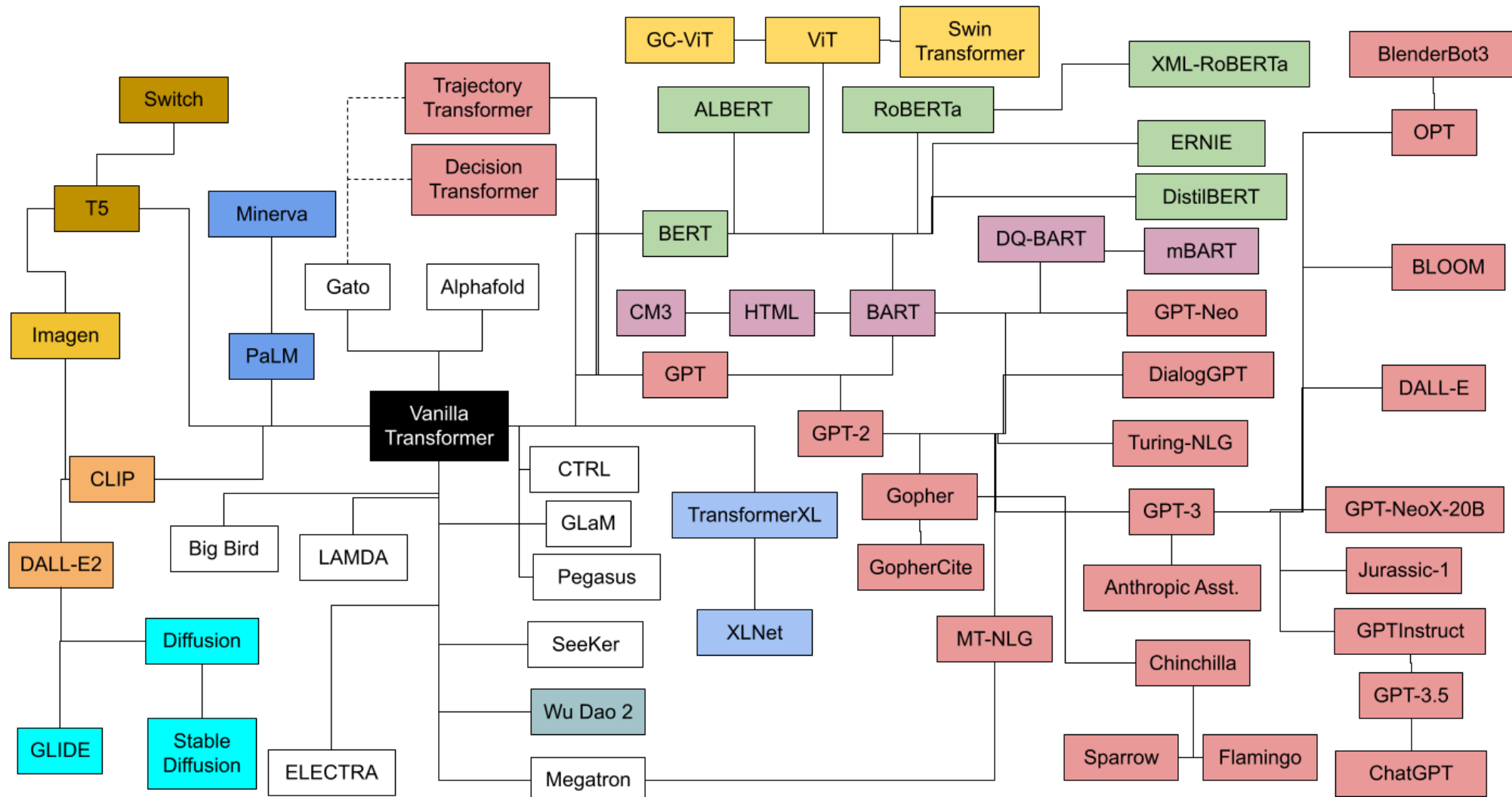
*https://openreview.net/pdf?id=i_Q1yrOegLY

**<https://arxiv.org/pdf/2205.13504.pdf>

Приложение: Хронология развития трансформеров



Приложение: Дерево связей



Как использовать?

Большой список доступных моделей и удобный интерфейс - библиотека HuggingFace Transformers 🤗

Методы дообучения LLM

Методы дообучения LLM

Fine-tuning (тонкая настройка) — это процесс дообучения уже существующей большой языковой модели (LLM) на специфических данных, чтобы адаптировать её к выполнению конкретной задачи или улучшить её точность в определённой области. Вместо того чтобы обучать модель с нуля (что требует огромных вычислительных ресурсов и данных), fine-tuning позволяет взять предобученную модель и адаптировать её под свои задачи с меньшими затратами.

Методы дообучения LLM

Fine-tuning (тонкая настройка) — это процесс дообучения уже существующей большой языковой модели (LLM) на специфических данных, чтобы адаптировать её к выполнению конкретной задачи или улучшить её точность в определённой области. Вместо того чтобы обучать модель с нуля (что требует огромных вычислительных ресурсов и данных), fine-tuning позволяет взять предобученную модель и адаптировать её под свои задачи с меньшими затратами.

Как это работает?

Предобученная модель: Модель уже обучена на большом корпусе текстов (например, модель GPT-2, GPT-3).

Данные для тонкой настройки: Составляется датасет, содержащий текст, релевантный задаче (например, вопросы и ответы, техническая документация и т.д.).

Процесс обучения: Модель обучается на новом наборе данных, используя предварительно выученные представления. Это ускоряет процесс и снижает требования к вычислительным ресурсам.

Оценка и корректировка: После обучения модель тестируется на отдельном наборе данных, чтобы убедиться, что она выполняет задачу на требуемом уровне.

Методы дообучения LLM

Традиционный метод **Fine-tuning**, при котором настраиваются все параметры предварительно обученной модели, становится непрактичным и вычислительно дорогостоящим при работе с современными моделями LLM

Методы дообучения LLM

Традиционный метод **Fine-tuning**, при котором настраиваются все параметры предварительно обученной модели, становится непрактичным и вычислительно дорогостоящим при работе с современными моделями LLM

PEFT Parameter-Efficient Fine-Tuning

PEFT (Parameter-Efficient Fine-Tuning) представляет собой эффективный подход, позволяющий не терять производительность при тонкой настройке модели, снижая при этом требования к памяти и вычислительным мощностям.

PEFT - это метод fine-tune, который позволяет улучшить результаты предварительно обученных языковых моделей при выполнении определенных задач. Его идея заключается в том, чтобы обучить небольшое подмножество параметров предварительно обученной модели LLM, оставляя большую часть замороженными.

Аддитивный метод в контексте машинного обучения и адаптации моделей означает подход, при котором изменения или адаптации к существующей модели добавляются **дополнительно**, а не заменяют оригинальные параметры модели. Это позволяет сохранить исходные знания модели и аккуратно добавлять новые, минимизируя вероятность утраты старых знаний.

Методы дообучения LLM

Преимущества PEFT

Ускорение времени обучения: PEFT позволяет сократить количество времени, затраченное на обучение, благодаря fine-tune небольшого количества параметров, а не всей модели.

Снижение затрат на вычисления и хранение: PEFT выполняет тонкую настройку только небольшого подмножества параметров, что значительно уменьшает затраты на вычисления и хранение и снижает требования к оборудованию.

Меньший риск переобучения: Благодаря замораживанию большей части параметров предварительно обученной модели мы можем избежать переобучения на новых данных.

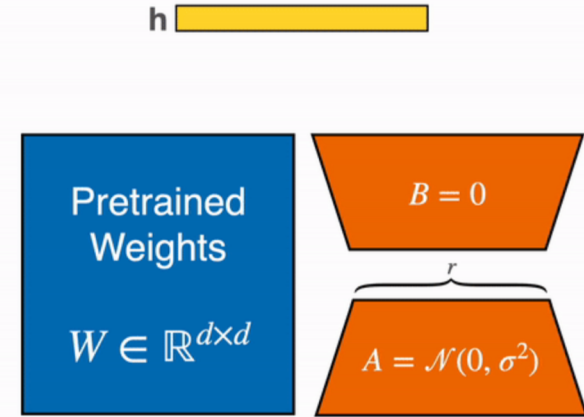
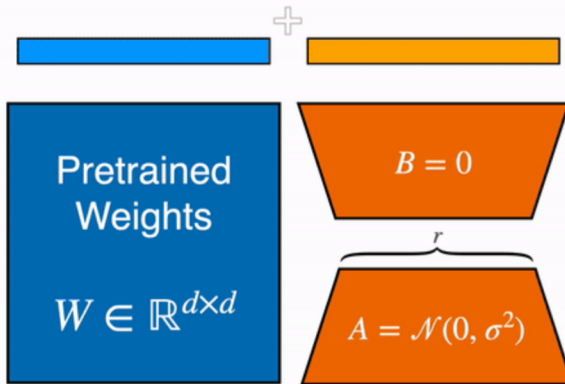
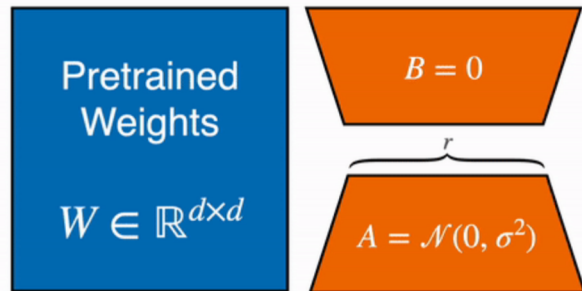
Преодоление катастрофического забывания: С помощью PEFT модель может адаптироваться к новым задачам, сохраняя ранее полученные знания благодаря заморозке большинства параметров.

Удобство развертывания: Контрольные точки, созданные при PEFT, компактнее, чем при традиционной тонкой настройке, что делает их развертывание и перенос на другие устройства более простым, поскольку они требуют меньше места для хранения и могут загружаться быстрее.

Методы дообучения LLM

LoRA

Метод LoRA (Low-Rank Adaptation) был разработан в 2021 году и представлен в данной [статье](#). Его создатели были вдохновлены данной научной [работой](#), в которой авторы отмечают, что, хотя LLM имеют миллионы или даже миллиарды параметров, они имеют низкую "внутреннюю размерность" (intrinsic dimension) при адаптации к новой задаче. Т. е. большинство параметров являются избыточными. Из чего можно сделать вывод, что матрицы можно представить пространством меньшей размерности, сохраняя при этом большую часть важной информации.



Создатели LoRA предположили, что изменение весов при fine-tuning модели имеет низкий "внутренний ранг" (intrinsic rank). Идея данного метода заключается в том, что для предварительно обученной матрицы весов мы представляем её обновление двумя меньшими матрицами, полученными путем низкоранговой аппроксимации. Эти матрицы мы тренируем при обучении, а исходную матрицу весов замораживаем. Затем для получения окончательного результата мы объединяем исходные и обученные веса.

Методы дообучения LLM

Adapters также является аддитивным методом. Он похож на метод Prefix tuning, только в этом случае мы добавляем не префикс, а адаптеры. Авторы данного метода ([статья](#)) предложили следующую структуру:

Внутри адаптера исходные d -мерные признаки сначала проецируются в меньшее измерение m , затем применяется нелинейность, а после снова проецируются в d -мерное измерение. Также здесь присутствует skip connection.

Как работает Adapter Tuning

Добавление адаптеров: Адаптеры — это небольшие обучаемые слои, которые добавляются между слоями предобученной модели. Эти слои обычно имеют небольшое количество параметров по сравнению с весами самой модели. Адаптеры обычно работают путем выполнения незначительных трансформаций входных данных на каждом слое, что позволяет модели адаптироваться к новой задаче или домену.

Параллельное использование: Основная модель (например, GPT-3 или BERT) остаётся фиксированной, и её веса не обновляются. Вместо этого адаптеры добавляются в виде дополнительных слоёв, которые обучаются на данных для конкретной задачи. Эти адаптеры могут быть разных типов, например, они могут работать как сети с пропуском (skip networks), которые вносят минимальные изменения в оригинальные слои.

